

ROS 2와 Raspberry Pi Pico 2 W로 만드는 micro-ROS 로봇

Embedded C · FreeRTOS · micro-ROS로 배우는
고정형 로봇 · 이족보행형 로봇
설계와 제작



집필 중 · 1~3장 무료 샘플

도서 정보

ROS 2와 Raspberry Pi Pico 2 W로 만드는 micro-ROS 로봇

Embedded C·FreeRTOS·micro-ROS로 배우는 고정형 로봇·이족보행형 로봇 설계와 제작

집필 중 · 1~3장 무료 샘플

최수길 지음 · 마담

이 PDF는 집필 중인 책의 1~3장 공개 샘플입니다. 본문은 2026년 9월 30일 원고를 기준으로 합니다. 전체 구성은 6부 24장이며, 다른 장을 가리키는 설명은 전체 책의 학습 흐름을 안내합니다. 4~24장 본문과 전체 전자책은 이 샘플에 포함하지 않았습니다.

공개 샘플만으로 따라 할 수 있도록 실습 파일 안내를 조정했습니다. 1장의 양식은 본문에서 복사하고, 3장의 두 C 예제는 인쇄된 전체 코드를 직접 파일로 저장하여 PC에서 실행합니다. 별도 비공개 저장소나 전체 예제 묶음을 받을 필요가 없습니다.

- 현재 상태: 24장 본문 초안, 기술·출판 교정 진행 중
- PC 논리 검사와 일부 ROS 2 호스트 검증 기록이 있으며, 교육용 통합 펌웨어와 실물 로봇 검증은 아직 완료되지 않았습니다.
- ISBN과 정식 발행일은 미정입니다. 샘플 기준일은 정식 발행일이 아닙니다.
- 최신 목차·진행 안내: [공개 실습 안내](#)
- 책 소개: [마담](#)

저작권 © 최수길. 이 샘플 PDF는 무료로 열람·다운로드할 수 있습니다. 별도 허락 없는 재배포·수정·상업적 이용을 허용하는 라이선스를 부여하지 않습니다. 인용은 관계 법령이 허용하는 범위에서 출처를 표시해 주세요.

본문·제목에는 Noto Sans KR, 코드에는 DejaVu Sans Mono를 내장했습니다. 글꼴의 저작권·라이선스 고지는 내장 글꼴에 유지했습니다. 표지와 작업대 개념 삽화는 생성형 이미지이며, 실제 부품·배선·제작 도면을 대신하지 않습니다.

목차

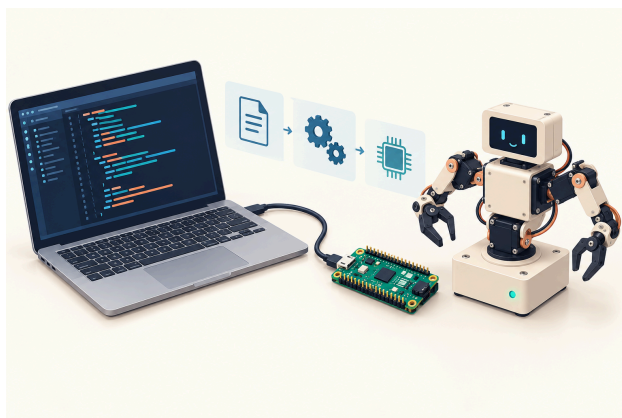
1~3장 공개 샘플

- 1장. 내가 만들 로봇의 임무를 정한다
- 2장. Pico 2 W 개발 환경과 재현 가능한 버전
- 3장. 로봇 제어에 필요한 Embedded C

01

CHAPTER

내가 만들 로봇의 임무를 정한다



학습 목표

- 입력·판단·동작·표시의 관계를 설명한다.
- 고정형과 이족형의 제작 차이를 구분한다.

이 장의 학습 흐름

STEP 01

임무 정의

STEP 02

책임 나누기

STEP 03

완료 조건

완료 기준

- 사용자와 기대 행동을 한 문장으로 적는다.
- 정지·통신 중단 시나리오를 임무 카드에 남긴다.

이번 장의 결과물 임무 카드와 시스템 스케치

책상 위에 작은 로봇이 놓여 있다고 생각해 보자. 손가락으로 센서를 건드리면 로봇이 한쪽 팔을 들어 인사하고, 화면에는 짧은 문장이 나타난다. 컴퓨터에서 명령을 보내면 두 팔을 움직이고, 동작을 마치면 다시 기다린다. 겉으로 보면 간단한 반응이지만, 그 안에서는 입력을 읽고, 할 일을 결정하고, 모터에 신호를 보내고, 현재 상태를 표시하는 일이 이어진다.

이번에는 두 발로 서 있는 로봇을 떠올려 보자. 네 개의 작은 모터가 다리와 발의 자세를 바꾸며 한 걸음을 만든다. 컴퓨터에서 전진을 요청할 수도 있지만, 매 순간의 관절 움직임을 컴퓨터의 무선 응답만 기다리며 처리할 수는 없다. 로봇 안에서는 움직임을 이어 가고, 새 명령과 정지 요청을 확인해야 한다.

이 책에서는 이처럼 작지만 여러 기능이 연결된 로봇을 만든다. Raspberry Pi Pico 2 W로 센서와 모터를 다루고, FreeRTOS로 여러 작업을 함께 진행하며, micro-ROS로 PC의 ROS 2 프로그램과 데이터를 주고받는다. 마지막에는 고정형 로봇 또는 이족보행형 로봇 중 하나를 조립하고, 자신의 요구에 맞게 동작을 바꾸어 본다.

처음부터 모든 기술을 알아야 하는 것은 아니다. 지금 필요한 첫 질문은 “어떤 라이브러리를 설치할까?”보다 “누가, 언제, 무엇을 요청하면 로봇이 어떻게 반응해야 할까?”에 가깝다. 만들 행동을 분명하게 정하면 필요한 부품, 프로그램의 역할, 시험할 항목도 차례로 정할 수 있다.

이 장에서는 보드에 전원을 연결하거나 펌웨어를 올리지는 않는다. 종이와 펜 또는 문서 편집기로 **로봇의 임무 카드와 시스템 스케치**를 작성한다. 이 두 가지 문서는 뒤의 장에서 부품을 연결하고 코드를 작성할 때 계속 펼쳐 보게 될 첫 설계도다.

시작 전에: 알면 도움이 되는 기초, 몰라도 시작할 수 있는 길

이 책은 로봇을 만들면서 임베디드 제어와 ROS 2 연결을 함께 익히는 책이다. 기본적인 C 문법과 터미널 사용에 익숙하다면 시작할 수 있다. 여기에 ATmega128 같은 마이크로컨트롤러로 간단한 입출력과 통신을 실습한 경험이 더해지면, 함수 호출이 실제 하드웨어 동작으로 이어지는 과정을 이해하기가 한결 쉬워진다.

미리 공부하거나 책과 함께 보강하면 좋은 내용은 다음과 같다.

- **레지스터와 비트 연산:** 주변장치의 설정·상태·데이터 레지스터를 구분하고, 필요한 비트를 설정하거나 읽어 입력과 출력을 제어하는 경험.
- **직렬 통신:** UART에서 송수신 설정과 바이트 전달을 익히고, SPI·I²C에서는 클록·장치 선택·주소 등 통신 방식에 따라 달라지는 약속을 이해하는 경험.
- **데이터 표현과 변환:** 2진수·16진수, 정수·문자·바이트의 차이를 구분하고, 수신한 바이트를 사용할 값으로 해석하거나 아날로그 입력을 ADC로 디지털 값으로 바꾸는 경험.
- **시간과 사건 처리:** 타이머·인터럽트의 기본 역할을 알고, 계속 입력을 확인하는 방법과 사건에 반응하는 방법을 비교해 본 경험.

여기서 사용하는 용어는 **레지스터(register)**다. 레지스터에 값을 쓰는 일은 단순히 숫자를 저장하는 것에 그치지 않고, 주변장치의 동작을 설정하거나 데이터를 전달하는 의미를 가질 수 있다. 어떤 비트가 무엇을 뜻하는지는 해당 마이크로컨트롤러의 문서를 보고 확인한다. 이런 기초를 알면 뒤에서 Pico SDK의 함수를 사용할 때도 무엇을 설정하고 어떤 결과를 읽는지 생각할 수 있다.

추천하는 참고 도서는 허경용의 『**ATmega128로 배우는 마이크로컨트롤러 프로그래밍**』(제이펍)이다. 마이크로컨트롤러의 구조와 레지스터, C의 비트 연산, 입출력·직렬 통신·변환·타이머·인터럽트 등 기초를 차근차근 보강하는 용도로 활용하자. **출판사 도서 안내**에서 목차를 확인하고 필요한 주제를 골라 읽어도 좋다.

이 내용을 모두 선행 학습해야 다음 장으로 넘어갈 수 있는 것은 아니다. ATmega128 경험이나 참고 도서의 완독을 필수로 요구하지 않는다. 처음 접하는 독자는 이 책의 흐름을 따라가면서 C와 비트·바이트 개념, 장치 입출력, 통신을 필요한 단계에서 보강하면 된다. 반대로 이미 경험이 있다면 익숙한 개념이 Pico와 FreeRTOS·micro-ROS에서 어떻게 이어지는지 비교해 보자. ATmega128과 Pico 2 W는 하드웨어와 개발 환경이 다르므로, 참고 도서의 레지스터 이름이나 코드를 그대로 옮기기보다 원리를 연결하는 것이 목적이다.

학습 목표와 완료 기준

이 장을 마치면 다음 질문에 자신의 말로 답할 수 있다.

- 작은 로봇을 입력·판단·동작·상태 확인으로 나누면 무엇이 보이는가?
- PC와 Pico 2 W는 어떤 일을 나누어 맡는가?
- ROS 2, micro-ROS, FreeRTOS는 각각 어떤 문제를 다루는가?
- 고정형 로봇과 이족보행형 로봇은 무엇을 공유하고 어디에서 달라지는가?
- “잘 움직인다”를 다른 사람도 확인할 수 있는 조건으로 어떻게 바꾸는가?

완료 기준

1. 만들 로봇의 사용자·사용 상황·입력·기대 행동을 한 문장으로 적는다.
2. PC·Agent·Pico·센서·출력 장치·전원을 표시한 시스템 스케치를 그리고, 화살표에 전달할 정보를 적는다.
3. 고정형 로봇 또는 이족보행형 경로를 선택하고, 필수 기능 세 가지와 이번엔 만들지 않을 기능을 구분한다.
4. 정상 동작·로컬 정지·통신 중단의 세 가지 시나리오와 확인 방법을 임무 카드에 적는다.

이 장의 완료는 로봇을 실제로 움직였다는 뜻이 아니다. 다른 사람이 자신의 설계를 읽고 무엇을 만들고 어떻게 확인할지 이해할 수 있다면 첫 단계를 마친 것이다.

1.1 로봇은 몸체와 프로그램이 함께 만드는 결과다

버튼을 누르면 모터가 움직이는 실습을 해 본 독자라면, 로봇 만들기를 부품 연결과 코드 작성의 합으로 생각하기 쉽다. 실제 제작에서는 그 사이에 몇 가지 약속이 더 필요하다. 버튼을 한 번 눌렀을 때 한 번만 움직일지, 계속 누르면 동작을 반복할지, 움직이는 동안 다시 눌렀을 때 어떻게 할지도 정해야 한다.

팔이 닿으면 안 되는 위치나 모터가 움직일 수 있는 범위는 몸체의 모양에 따라 달라진다. 같은 프로그램이라도 서보 호를 다른 각도로 체결하면 팔의 시작 자세가 달라질 수 있다. 전원이 충분하지 않으면 논리적으로 올바른 프로그램도 기대한 동작을 만들지 못한다.

따라서 이 책에서 로봇은 **입력을 받아 정한 규칙에 따라 물리적으로 반응하고, 그 결과를 확인할 수 있는 시스템**으로 다룬다. 사람처럼 생겼는지, 스스로 먼 곳을 이동하는지는 첫 기준이 아니다. 센서 하나와 움직이는 관절 하나를 가진 작은 장치도 설계·제어·검증을 배우는 출발점이 될 수 있다.

입력, 판단, 동작, 상태 확인

터치하면 인사하는 로봇을 다음과 같이 나누어 보자.

- **입력:** 터치 센서가 눌렀는지 읽는다.
- **판단:** 지금 대기 중이며 정지 상태가 아니라면 인사를 시작한다.
- **동작:** 정한 순서에 따라 팔의 목표 자세를 바꾼다.
- **상태 확인:** 동작 중·완료·정지 상태를 화면이나 PC에 알린다.

여기서 판단은 반드시 AI를 뜻하지 않는다. “대기 중에 입력이 들어오면 인사한다”는 조건문도 판단이다. 처음에는 사람이 이해하고 시험하기 쉬운 규칙으로 충분하다. 복잡한 인식 기능을 더하기 전에 이 작은 흐름이 일관되게 동작해야 한다.

작은 로봇의 행동을 네 가지로 나누기

예시: 터치하면 오른팔로 인사하고, 준비 상태로 돌아오는 고정형 로봇

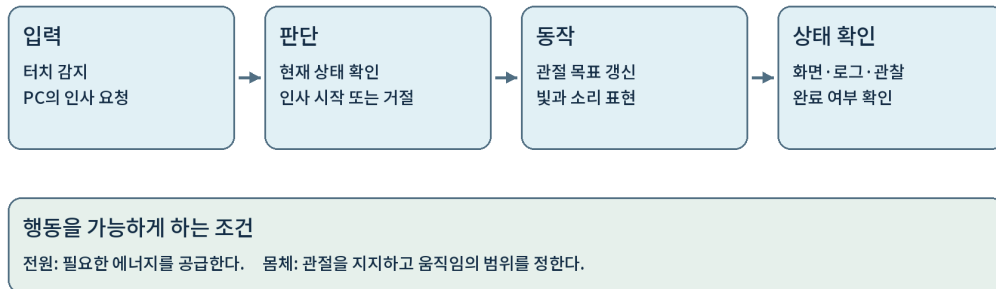


그림 1-1. 입력·판단·동작·상태 확인을 나누면 각 기능이 주고받는 정보가 보인다. 전원과 몸체는 이 흐름 전체의 조건이다.

그림의 상태 확인이 곧 실제 관절 각도의 측정을 뜻하지는 않는다. 일반적인 위치 제어 서보에 목표 각도를 보냈더라도, 별도 위치 센서가 없다면 실제로 그 위치에 도달했는지 직접 읽지 못할 수 있다. 프로그램이 “동작 완료”라고 표시했을 때도 **명령 순서가 끝났다는 뜻인지, 센서로 동작 결과까지 확인했다는 뜻인지**를 구분해야 한다.

이 차이는 작아 보이지만 중요하다. 팔이 몸체에 걸쳐 움직이지 않았는데 명령만 끝났다고 성공으로 기록하면, 문제를 찾을 기회를 놓친다. 이 책에서는 코드의 상태와 사람이 관찰한 실제 동작을 함께 확인한다.

1.2 기능이 늘어나면 기다리는 방법이 중요해진다

인사 동작만 있을 때는 팔을 올리고 잠시 기다린 다음 다시 내리는 순서로 작성할 수 있다. 그런데 그 기다리는 동안 정지 버튼을 읽어야 한다면 어떻게 될까? 같은 흐름이 긴 시간 멈춰 있다면 정지 입력을 뒤늦게 처리할 수 있다.

여기에 화면 갱신과 Wi-Fi 통신이 더해진다. 화면에 긴 문장이 흘러가는 동안에도 터치를 읽어야 하고, 네트워크 응답이 늦어도 이미 시작한 동작의 상태를 관리해야 한다. 여러 기능을 넣는 것만큼 **한 기능이 다른 기능을 얼마나 기다리게 하는지**가 중요해진다.

아래 두 방식은 코드가 아니라 동작을 설명한 설계 예시다.

순서대로 오래 기다리는 방식
팔 올리기 → 기다리기 → 팔 내리기 → 터치 확인

짧게 확인하며 진행하는 방식

현재 시간 확인 → 필요한 만큼 관절 갱신
→ 터치와 정지 요청 확인 → 표시 갱신 → 다시 확인

두 번째 방식에서는 인사 동작이 아직 끝나지 않았어도 다른 입력을 살펴볼 수 있다. 이후 FreeRTOS를 배우면 이런 작업을 태스크로 나누고, 언제 실행하고 언제 양보할지 정리한다. 다만 처음부터 태스크를 많이 만드는 것이 목표는 아니다. 먼저 통신 없이 반응하는 작은 프로그램을 만들고, 기능을 나누어야 하는 이유를 경험한 뒤 적용한다.

또한 “동시에 진행된다”와 “정확히 같은 순간에 실행된다”는 다르다. 한 코어가 여러 작업을 짧게 번갈아 실행해도 사용자에게는 함께 진행되는 것처럼 보일 수 있다. 두 코어를 사용하면 실제로 겹쳐 실행되는 작업도 생기지만, 공유하는 장치와 데이터는 더 신중하게 다뤄야 한다. 이 차이는 11장과 12장에서 확인한다.

1.3 PC와 Pico 2 W의 역할을 나눈다

Raspberry Pi라는 이름 때문에 Pico 2 W를 작은 Linux 컴퓨터로 생각할 수 있다. Pico 2 W는 Raspberry Pi 5처럼 일반적인 Linux 환경에서 여러 응용 프로그램을 실행하는 보드와 쓰임이 다르다. RP2350 마이크로컨트롤러를 중심으로 센서와 주변장치를 제어하는 보드이며, 이 책에서는 C로 작성한 펌웨어를 올려 사용한다.

펌웨어는 보드에서 실행되어 하드웨어의 동작을 담당하는 프로그램이다. 독자는 PC에서 소스를 편집하고 빌드한 결과를 Pico에 올린다. 프로그램을 만드는 장소와 실제 장치를 제어하는 장소가 다르다는 점을 기억하자.

임베디드 시스템은 장치 안에서 맡은 일을 한다

임베디드 시스템을 처음에는 “장치 안에 들어 있는 작은 컴퓨터”라고 떠올려도 좋다. 다만 핵심은 크기가 아니라 **특정 기능을 수행하도록 제품이나 장치에 내장되어 있다는 것이다**. 로봇의 관절 제어부, 세탁기의 제어부, 센서의 측정 장치가 여기에 해당한다. 하드웨어와 소프트웨어가 함께 기능을 만든다.

Pico 2 W 같은 마이크로컨트롤러 보드에서는 전원·배선·센서·모터 구동 회로와 펌웨어가 서로 맞아야 한다. 전압과 신호를 맞추는 회로 설계, 입력을 읽고 출력을 갱신하는 펌웨어 프로그래밍, 제한된 메모리와 실행 시간을 배분하는 일이 함께 필요하다. 모터를 보드의 신호 핀에 직접 연결해 구동한다는 뜻은 아니다. 부품에 맞는 전원과 구동 회로를 확인하는 과정은 5장에서 다룬다.

이 책에서는 기성 Pico 보드와 모듈을 연결하면서 임베디드 시스템의 기본을 배운다. 처음부터 마이크로컨트롤러 칩을 넣은 전용 기판을 설계할 필요는 없다. 대신 부품 연결과 코드가 실제로 어떤 행동을 만드는지 설명할 수 있어야 한다.

가까이에서 제어하면 줄일 수 있는 기다림

Pico 2 W는 센서 입력과 관절 명령 갱신을 로컬에서 짧은 주기로 처리하도록 구성할 수 있다. PC의 판단을 매번 기다리는 구조에서는 센서 정보를 보내고 명령을 돌려받는 통신 시간에 더해, PC 운영 체제와 프로그램의 실행 대기가 응답에 영향을 준다. 특히 무선 통신은 혼잡과 재전송에 따라 지연이 달라질 수 있다.

중요한 점은 PC의 계산 능력이 부족하다는 것이 아니다. **로봇이 지금 처리해야 할 일이 통신과 다른 프로그램의 사정에 얼마나 의존하는가**가 문제다. 예를 들어 팔을 움직이는 동안 정지 입력이 들어오면, 화면 처리나 PC의 다음 응답보다 그 입력을 먼저 살펴볼 수 있어야 한다. 그래서 보드 안에서 입력을 확인하고 동작을 갱신하는 흐름을 둔다.

많은 로봇은 이러한 제어를 마이크로컨트롤러, 모터 드라이브 안의 제어기, 전용 제어 장치 등에 맡긴다. 모든 로봇에 별도의 Pico 같은 보드가 반드시 있어야 하는 것은 아니다. 이 책에서는 Pico가 로컬 제어를 맡고 PC가 행동 선택과 상태 확인을 맡는 구조로 그 이유를 경험한다.

PC는 사용자의 명령을 받고, 데이터를 보기 좋게 표시하고, 여러 로봇 프로그램을 연결하는 데 적합하다. Pico는 센서 입력을 읽고 관절·버저·LED 같은 장치를 가까이에서 제어한다. 모든 기능을 무조건 어느 한쪽에 넣기보다, 이 프로젝트에서 누가 책임져야 하는지 정한다.

구분	PC에서 맡을 일	Pico 2 W에서 맡을 일
요청	사용자가 행동을 선택한다	요청을 검사하고 실행 여부를 정한다
움직임	인사·전진 같은 목표를 보낸다	정한 시간에 관절 명령을 갱신한다
상태	로그와 결과를 보여 준다	입력·실행·정지 상태를 만든다
통신 중단	연결 문제를 사용자에게 알린다	정해 둔 동작 중단 정책을 따른다

이 표는 이 책에서 채택할 책임 분리다. 모든 로봇에 적용되는 유일한 구조는 아니다. 예를 들어 간단한 터치 반응은 PC 없이도 Pico 안에서 실행할 수 있다. 반면 여러 로봇의 상태를 한 화면에서 보는 기능은 호스트 쪽에 두는 편이 자연스럽다.

호스트가 없어도 필요한 기능

PC를 끄면 화면은 볼 수 없지만, 로봇의 로컬 정지 입력까지 사라져서는 안 된다. 무선 연결이 끊겼을 때도 로봇 안에서는 새 명령이 오지 않는다는 사실을 판단하고 정한 상태로 전환해야 한다.

그래서 이 책은 첫 로봇 반응을 로컬에서 만든다. 이후 통신을 더하더라도 입력·동작·정지의 기본 책임을 모두 네트워크 너머로 넘기지 않는다. 이 구조는 문제를 찾을 때도 도움이 된다. 로컬에서 팔이 움직인다면 먼저 전원·배선·관절 제어를 확인한 셈이고, 그다음에는 명령이 어디까지 도착하는지 살펴볼 수 있다.

Pico 2 W는 여러 선택지 중 이 책의 기준이다

로컬 제어를 맡을 보드는 Pico 2 W만 있는 것이 아니다. ESP32 계열, STM32 Nucleo, Teensy 등도 목적과 개발 환경에 따라 후보가 된다. 이 책은 배선과 코드를 같은 조건에서 설명하기 위해 Pico 2 W를 기준으로 삼는다. 2장에서 선택 이유와 공식 하드웨어 사양, Pico SDK, 다른 보드의 가격·특징을 비교한다.

중요한 것은 제품 이름보다 임무에 필요한 핀·전원·메모리·통신과 개발 지원이다. 입력을 검사하고 정지를 우선하는 설계는 다른 보드에서도 활용할 수 있으며, 핀 제어와 드라이버·빌드 환경은 새 보드에 맞춰 바꾼다. 처음에는 기존 보드로 작은 성공을 쌓고, 이후 자신의 요구에 맞는 보드를 설명하고 선택하는 것을 목표로 하자.

1.4 ROS 2, micro-ROS, FreeRTOS는 서로 다른 일을 한다

세 이름을 처음 만나면 비슷한 소프트웨어를 세 번 배우는 것처럼 느껴질 수 있다. 역할을 한 문장씩 연결해 보자.

ROS 2는 로봇 소프트웨어의 기능과 데이터를 연결하는 기반이고, micro-ROS는 마이크로컨트롤러를 그 연결에 참여시키며, FreeRTOS는 보드 안에서 실행할 작업의 순서를 관리한다.

ROS 2: 프로그램 사이의 데이터 약속

ROS 2에서는 한 가지 논리적 책임을 맡는 단위를 Node라고 부른다. 예를 들어 PC에서 행동을 선택하는 Node와 로봇 상태를 표시하는 Node를 나눌 수 있다. Node가 반드시 파일 하나, 프로세스 하나 또는 보드 하나와 일치하는 것은 아니다.

Topic은 이름과 메시지 타입을 기준으로 데이터를 전달하는 통로다. 데이터를 발행하는 쪽은 Publisher, 받는 쪽은 Subscriber다. 이 책에서는 센서 상태와 장치 명령을 주고받는 데 이 개념을 사용한다. 지금 명령어나 API를 외울 필요는 없다. **누가 어떤 데이터를 만들고 누가 사용하는가**를 먼저 생각하면 된다.

앞으로 “행동 요청”이나 “완료 상태”라는 말을 사용할 때도 데이터의 의미를 먼저 설계한다. 이후 장에서 메시지 타입과 통신 방식을 정한다. 여기서 설명하는 인사 요청이 이미 구현된 Action이나 특정 이름의 Topic이라는 뜻은 아니다.

micro-ROS: 작은 보드와 ROS 2를 연결하기

Pico에 PC용 ROS 2 환경을 그대로 설치하는 대신, 마이크로컨트롤러에 맞춘 micro-ROS 클라이언트를 사용한다. 보드 쪽에서는 C 기반 API와 rclcl를 통해 노드·발행·구독·콜백 실행을 구성할 수 있다. 세부 구조는 13장 이후의 실제 통신 실습에서 다룬다.

이 책의 기본 구성에서는 PC에서 **micro-ROS Agent**도 실행한다. Agent는 MCU 쪽의 micro-ROS 통신을 일반 ROS 2 시스템과 연결한다. 먼저 직렬 연결로 왕복을 확인하고, 이후 Wi-Fi의 UDP 경로로 바꾼다. 연결 수단이 바뀌어도 어떤 명령과 상태를 주고받을지는 유지하는 방향으로 설계한다.

Agent를 모든 ROS 2 Node가 반드시 거쳐야 하는 중앙 서버로 이해하면 안 된다. 여기서는 마이크로컨트롤러를 연결하기 위해 사용하는 구성 요소다. PC 안의 ROS 2 프로그램끼리 데이터를 주고받는 구조 전체를 Agent가 대신하는 것은 아니다.

아두이노와 USB 통신에서 한 걸음 더 나아가기

아두이노를 사용해 본 독자라면 PC와 보드를 USB로 연결하고, PC에서 보낸 문자를 보드가 읽어 LED나 모터를 제어한 경험을 떠올려 보자. USB 시리얼로 충분한 실습도 많다. 이때는 양쪽 프로그램이 “어떤 문자가 어떤 명령인지”, “센서 값은 어떻게 구분할지” 같은 약속을 직접 맞춘다.

로봇 기능이 늘어나면 센서 값을 화면·기록 프로그램·행동 판단 프로그램에서 함께 쓰고 싶어진다. 프로그램마다 전용 시리얼 해석 코드를 작성하면 데이터 형식이 바뀔 때 여러 곳을 수정해야 한다. micro-ROS를 쓰는 이유는 **보드의 데이터와 명령을 ROS 2의 Node·Topic·메시지 타입이라는 공통 방식으로 다루기 위해서**다. Agent를 통해 연결하면 호스트에서 ROS 2 도구로 데이터를 확인하고 다른 Node와 연동할 수 있다.

이 책에서는 micro-ROS 클라이언트 라이브러리와 통신 처리 코드를 펌웨어에 포함한다. 그 옆에 독자가 작성한 센서·서보 제어 코드와 FreeRTOS 태스크가 함께 들어간다. micro-ROS가 서보 구동 방법이나 보행 알고리즘을 자동으로 만들어 주는 것은 아니다. **장치가 맡을 행동은 펌웨어가 구현하고, ROS 2와 주고받을 데이터의 연결은 micro-ROS가 돕는다.**

호스트는 PC일 수도 있고 Raspberry Pi 5처럼 Linux와 ROS 2를 실행하는 SBC(Single Board Computer, 단일 보드 컴퓨터)일 수도 있다. SBC는 여러 컴퓨터 구성 요소를 한 기판에 모은 형태를 가리킨다. 여기서 micro-ROS가 연결하는 핵심 대상은 Pico 같은 마이크로컨트롤러 쪽의 프로그램이다. SBC와 ROS 2를 서로 연결해야만 하는 별개의 대상으로 생각할 필요는 없다. PC나 SBC가 ROS 2 프로그램과 Agent를 실행하는 호스트가 된다.

통신을 더한 뒤에도 빠른 입력 확인과 관절 갱신은 로컬에 남긴다. micro-ROS는 분산된 기능을 연결하는 역할을 하며, 통신을 넣었다는 이유로 보드의 실시간 처리 책임이 사라지는 것은 아니다.

FreeRTOS: 보드 안의 실행 순서 관리

FreeRTOS는 작은 임베디드 시스템에서 태스크를 스케줄링하는 실시간 운영체제 커널이다. 입력 확인, 화면 갱신, 통신 처리 같은 일을 태스크로 나누고, 실행 가능한 작업 가운데 무엇을 먼저 실행할지 정하는 데 사용한다.

실시간이라는 말은 “무조건 가장 빠르다”는 뜻으로 받아들이지 말자. 정해진 시간 안에 필요한 일을 처리할 수 있도록 구조와 우선순위를 설계하는 일이 중요하다. FreeRTOS를 넣었다는 사실만으로 정지 반응 시간이나 통신 지연이 자동 보장되지는 않는다. 실제로 얼마나 걸리는지는 나중에 측정한다.

반응성과 실시간성을 따로 확인하는 이유

반응성은 입력이나 사건에 얼마나 빨리 반응하는지를 말한다. **실시간성**은 필요한 처리를 정한 마감 시간 안에 끝내는 성질이다. 평균적으로 빠르더라도 가끔 긴 지연이 생기면 관절 갱신 주기가 흔들리거나 정지 처리가 늦어질 수 있다. 계산 결과가 맞는 것과 필요한 때에 도착하는 것을 함께 확인해야 한다.

설명을 위한 가상의 목표로 “입력이 감지된 시점부터 20 ms 안에 정지 상태로 전환한다”를 생각해 보자. 대부분 2 ms에 처리하더라도 화면 갱신 중 한 번 80 ms가 걸렸다면 그 목표를 만족하지 못한다. 여기서 20 ms는 제품 요구나 실측값이 아니라 마감 시간을 이해하기 위한 예시다. 실제 목표는 몸체·속도·센서·제어 방식에 맞추어 정한다. 또한 소프트웨어의 정지 상태 전환과 모터가 물리적으로 멈추는 시간은 다르다.

고정형 로봇에서는 팔의 움직임과 정지 입력을 놓치지 않는 일이 중요하다. 이족보행형 로봇에서는 좌우 관절을 갱신하는 순서와 간격이 흐트러지면 의도한 자세와 지지가 달라질 수 있다. 반면 화면의 장식 효과는 조금 늦어져도 움직임만큼 큰 영향을 주지 않을 수 있다. 모든 기능에 같은 시간 요구를 적용하는 대신, 늦었을 때 생기는 결과를 기준으로 우선순위를 정한다.

시간 요구를 지키려면 짧은 제어 처리, 적절한 태스크 우선순위, 제한된 대기 시간, 공유 자원을 기다리는 시간까지 함께 설계해야 한다. 네트워크 응답이나 긴 화면 갱신을 기다리는 동안 중요한 제어가 막히지 않게 나누는 이유도 여기에 있다. FreeRTOS는 실행 순서·시간 관리·태스크 간 통신 수단을 제공하지만, 응용 프로그램의 마감 시간을 자동으로 증명해 주지는 않는다.

이 책에서는 “언제부터 언제까지 재는가”를 먼저 정하고, 통신·표시가 함께 동작하는 조건에서 처리 시간과 주기 변동을 측정한다. 몇 번 성공한 결과만으로 모든 상황의 시간을 보장했다고 말하지 않는다. 엄격한 시간 보장을 주장하려면 최악의 실행 시간과 대기 요인, 부하 조건까지 분석해야 한다. 먼저 독자는 자신이 정한 조건에서 시간 목표를 지키는지 확인하고 실패 기록을 남기는 법을 익힌다.

또한 ROS 2의 Node와 FreeRTOS 태스크는 다른 개념이다. 하나의 micro-ROS Node가 여러 Topic을 다룰 수 있고, 보드 안에서는 통신·표시·주변장치 작업을 별도 태스크로 실행할 수 있다. “Node 하나에 태스크 하나” 또는 “태스크 하나에 코어 하나”라는 대응을 정해 놓고 생각하지 않는다.

PC와 Pico가 나누어 맡는 책임

논리적인 역할을 나타낸 개념도 · Node, 태스크, 코어가 일대일로 대응한다는 뜻은 아니다

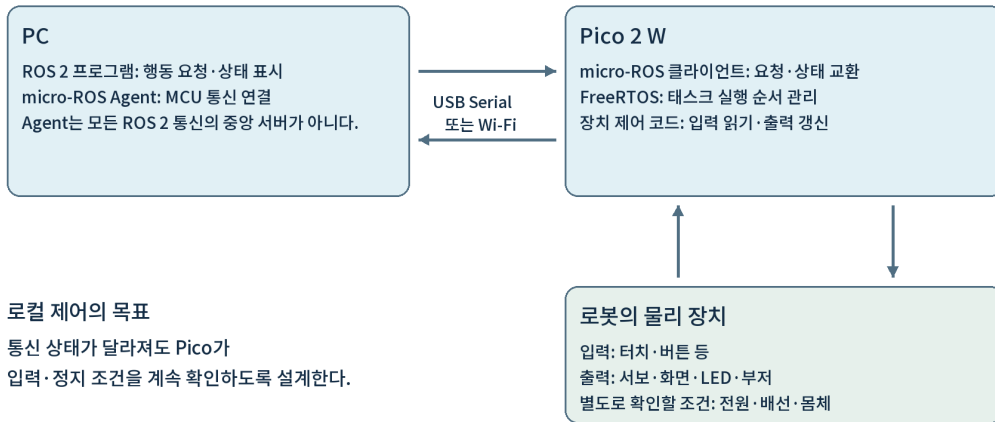


그림 1-2. PC의 ROS 2 프로그램과 Agent, Pico의 micro-ROS·FreeRTOS·장치 제어는 서로 다른 역할을 맡는다. 같은 PC에서 실행되는 프로그램도 역할에 따라 나누어 표현했다.

그림에서 PC가 하는 “인사 요청”과 Pico가 하는 “관절 갱신”은 다른 크기의 작업이다. 요청이 한 번 도착한 뒤에도 보드 안에서는 시간이 흐를 때마다 다음 자세를 계산하고 입력을 확인한다. 호스트는 그 진행 상태를 받을 수 있지만, 보드의 모든 세부 동작을 네트워크 한 번의 왕복에 묶지는 않는다.

1.5 두 가지 로봇, 하나의 학습 기반

이 책의 기본 제작 예제는 두 팔로 반응하는 **고정형 로봇**이다. 선택 제작 예제는 네 관절로 서고 걷는 **이족보행형 로봇**이다. 독자는 하나를 선택해도 된다. 공통 기초를 익힌 뒤 다른 경로를 추가로 제작할 수도 있다.

두 가지 로봇의 형태를 먼저 그려 보기

공통 제어 보드: Pico 2 W · 원 안의 숫자: 서보 관절 · 비올과 치수는 설명용

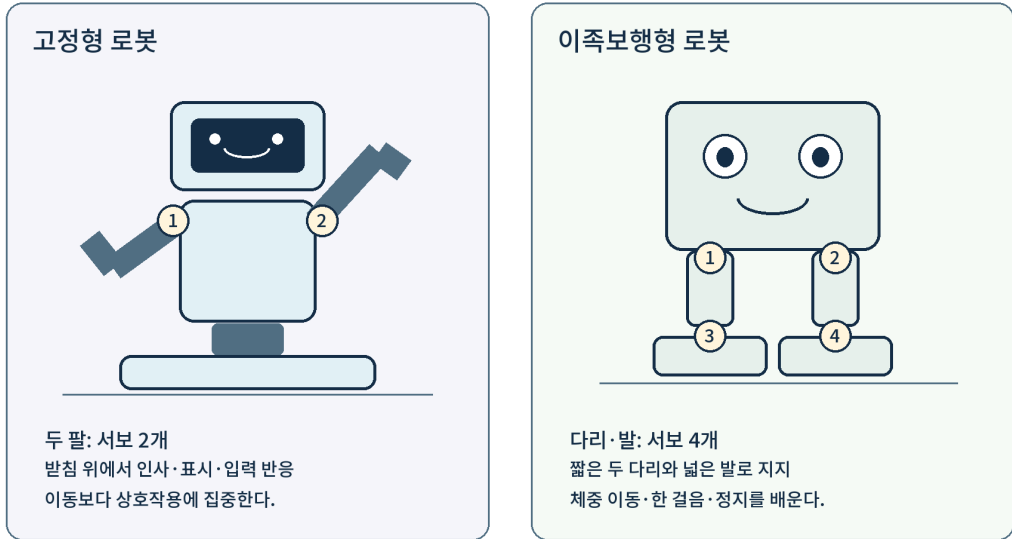


그림 1-3. 두 제작 경로의 외형 개념도. 고정형 로봇은 받침 위 몸체와 두 팔로 표현하고, 이족보행형 로봇은 상자 모양 몸체 아래 짧은 두 다리와 넓은 발로 자세를 바꾼다. 원·숫자는 관절 위치를 나타내며 치수나 제작 도면을 뜻하지 않는다.

이족보행형은 큰 몸체 아래 두 개의 짧은 다리와 넓은 발이 놓인 단순한 형태로 생각하면 된다. 좌우 다리와 발에 배치한 네 관절로 체중을 옮기며 걷는 구성을 목표로 한다. 고정형은 바닥을 이동하는 대신 안정된 받침 위에서 두 팔과 표시 장치로 반응한다. 눈처럼 보이는 도식의 표시는 외형을 이해하기 위한 표현이며, 카메라나 거리 센서를 필수로 넣는다는 뜻은 아니다.

고정형 로봇: 움직임과 표현을 연결한다

고정형 로봇 경로는 서보 두 개로 팔을 움직이고, 터치 센서·OLED·버저·네오픽셀로 사람과 상호작용하는 데 초점을 맞춘다. 터치에 반응해 인사하기, 숫자를 바꾸며 소리 내기, PC에서 보낸 짧은 문장을 화면에 표시하기 같은 행동으로 시작한다.

참고 저장소에는 이러한 장치 제어와 네 가지 동작 모드가 구현되어 있다. 그러나 기존 개발 코드를 옮기는 것만으로 교육용 로봇이 완성되는 것은 아니다. 새 책에서는 관절 보정, 잘못된 명령 처리, 로컬·원격 제어권, 정지와 재접속을 확인하며 예제를 재구성한다.

팔의 반응을 눈으로 쉽게 볼 수 있고 보행 균형을 다루지 않아도 되므로, 처음에는 입력과 출력의 연결에 집중하기 좋다. 대신 팔의 가동 범위와 몸체 간섭, 배선 여유를 살펴야 한다.

이족보행형: 자세가 한 걸음이 되는 과정을 배운다

이족보행형 경로는 네 개의 서보로 다리와 발의 자세를 바꾸는 간단한 이족 로봇을 목표로 한다. 관절이 각각 움직이는 데서 끝나지 않고, 몸체를 지지한 상태에서 무게를 옮기고 앞으로 나아가게 해야 한다.

학습 순서는 관절별 중립과 방향 확인, 서 있는 자세, 제자리 체중 이동, 한 걸음, 반복 전진과 정지다. “걷기 함수 하나를 호출한다”는 결과 뒤에 어떤 보정과 순서가 필요한지 살펴본다.

참고한 공개 이족보행 로봇 라이브러리는 Arduino C++ 환경을 대상으로 한다. 이 책의 이족보행형 경로는 Pico 2 W와 Pico SDK C를 기준으로 별도 검증할 대상이다. 기존 Arduino 예제를 그대로 복사하면 동작한다고 가정하지 않는다. 채택할 몸체와 코드의 출처·이용 조건도 함께 확인한다.

비교	고정형 로봇 경로	이족보행형 경로
주된 경험	터치에 반응하고 팔·문자·빛·소리로 표현	자세를 바꾸고 균형을 유지하며 전진
기본 관절	서보 2개	서보 4개
먼저 확인할 것	팔 방향·중립·몸체 간섭	네 관절 보정·발 접촉·지지 자세
결과 관찰	입력 뒤의 인사와 상태 표시	서기·전진·정지와 넘어짐 여부
공통 기반	전원·C·상태기계 ·FreeRTOS·micro-ROS	전원·C·상태기계 ·FreeRTOS·micro-ROS

OLED나 네오픽셀을 배우더라도 이족보행형 몸체에 모든 모듈을 넣을 필요는 없다. 먼저 버저와 상태 LED로 필요한 정보를 보여 주고, 임무에 도움이 될 때 표시 장치를 추가할 수 있다. 부품을 많이 넣는 것과 임무를 잘 수행하는 것은 같은 목표가 아니다.

현재 집필 단계에서 고정형 로봇의 참고 구현과 개발 기록은 존재하지만, 이 책의 교육용 펌웨어와 두 제작 경로의 실물 재현은 아직 검증 중인 대상이다. 특히 이족보행형의 Pico 펌웨어와 보행은 신규 개발 범위다. 이 장에서는 제작 방향과 합격 목표를 정한다.

1.6 좋은 임무는 관찰 가능한 행동으로 쓴다

“재미있는 로봇을 만든다”는 좋은 출발점이지만 제작 기준으로는 부족하다. 어떤 입력을 주어야 하는지, 언제 동작이 끝나는지, 무엇을 보고 성공이라고 할지가 빠져 있기 때문이다.

다음 두 문장을 비교해 보자.

처음 생각: 사람에게 반응하는 귀여운 로봇을 만든다.

첫 임무: 책상 위에서 사용자가 터치 입력을 한 번 주면, 로봇이 오른팔로 한 번 인사하고 대기 상태로 돌아온다.

두 번째 문장은 사용자, 장소, 입력, 동작, 종료 상태가 드러난다. 여기에 “동작 중 정지를 요청하면 어떻게 할 것인가?”를 더하면 첫 시험 시나리오까지 만들 수 있다.

먼저 작은 임무 하나를 완성한다

처음부터 음성 대화, 사람 인식, 자율 보행, 원격 앱을 모두 넣으면 실패했을 때 어디부터 확인할지 어려워진다. 한 가지 임무를 먼저 통과시킨 뒤 그 결과를 바탕으로 확장하자. 첫 임무는 다음 세 조건을 만족하면 좋다.

1. 사람이 같은 입력을 여러 번 줄 수 있다.
2. 결과를 눈·소리·로그 가운데 하나 이상으로 확인할 수 있다.
3. 실행 중 멈추고 초기 상태로 돌아가는 방법을 설명할 수 있다.

기능과 조건을 함께 적는다

“팔을 움직인다”는 기능이다. “지정한 범위 안에서 움직인다”, “정지 입력을 무시하지 않는다”, “통신이 복구되어도 이전 명령을 저절로 다시 실행하지 않는다”는 그 기능이 지켜야 할 조건이다.

처음에는 시간을 숫자로 정하기 어려울 수 있다. 측정하지 않은 응답 시간을 보장한다고 쓰기보다, **무엇을 측정하고 어느 단계에서 기준을 설정할지** 기록한다. 예를 들어 “정지 요청부터 동작 중단까지의 시간을 18장 통합 시험에서 측정하고 제작 승인 전에 허용값을 정한다”라고 쓸 수 있다. 미정 항목을 남기는 것과 확인하지 않은 값을 확정하는 것은 다르다.

고정형 로봇 임무 카드의 완성 예

다음 카드는 문서 작성 예시다. 실제 시연 결과나 이미 검증된 성능표가 아니다.

프로젝트 이름: 책상 위 인사 로봇

사용자와 상황: 실습자가 책상 위 로봇의 반응을 직접 확인한다.

한 문장 임무: 대기 중 터치 입력을 받으면 한쪽 팔로 한 번 인사하고, 상태를 표시한 뒤 다시 기다린다.

필수 기능 세 가지

1. 터치 입력으로 로컬 인사를 시작한다.
2. PC 요청으로 인사와 짧은 메시지 표시를 수행한다.
3. 로컬 정지와 명령 만료를 처리하고, 재연결 후 새 요청을 기다린다.

이번에 제외할 것: 사람 얼굴 인식, 음성 대화, 배터리 충전 회로, 이동 기능.

상태: 대기, 동작 중, 정지, 오류. Wi-Fi와 Agent 연결 상태는 별도로 기록한다.

관찰 방법: 팔의 실제 움직임, OLED의 상태 문구, PC의 요청·상태 로그를 대조한다.

아직 정할 것: 조립 후 관절 범위, 동작 시간, 정지 지연의 허용값, 최종 전원 부품.

여기서 정지 입력을 어느 센서에 할당할지와 기존 모드 전환 입력을 어떻게 구분할지는 후속 설계에서 정한다. 기존 고정형 로봇의 센서 번호에 새로운 정지 기능이 이미 있다고 읽지 않는다.

이족보행형으로 바꾸면 달라지는 것

같은 양식으로 “전진 요청을 받으면 짧은 거리를 걷고 정지한다”는 임무를 작성할 수 있다. 다만 관찰 항목은 달라진다. 팔이 올라갔지만 보는 대신 시작 자세, 이동거리, 발의 미끄러짐, 넘어짐, 정지 후 자세를 함께 본다.

이족보행형의 필수 기능을 **로컬 시작·정지, 원격 전진 요청과 상태 보고, 통신 중단 시 정지 정책**으로 정해 보자. 첫 제작에서는 점프나 빠른 춤 동작을 제외한다. 그러면 공통 통신 구조는 유지하면서, 몸체와 관절·동작 순서에 집중할 수 있다.

1.7 정지와 실패도 임무의 일부다

로봇이 잘 움직이는 장면만 떠올리면 정지는 나중에 추가할 작은 기능처럼 보인다. 그러나 실제 설계에서는 언제 무엇을 멈출지 먼저 정해야 한다. 팔을 움직이는 도중 몸체에 걸렸거나, PC가 꺼졌거나, 오래된 명령이 큐에 남아 있을 수 있기 때문이다.

세 가지 정지를 구별해 보자.

- **정상 종료:** 요청한 동작이 끝나 대기 상태로 돌아간다.
- **소프트웨어 정지:** 입력이나 명령 만료를 확인해 정해 둔 정지 상태로 전환한다.
- **전원 차단:** 소프트웨어와 별개로 전력 공급을 끊는다.

세 경우의 결과는 같지 않을 수 있다. 특히 서 있는 로봇은 서보 출력을 해제하면 지지를 잃고 넘어질 수 있다. 그래서 “모터를 끄면 항상 안전하다”는 한 문장으로 모든 몸체를 설명할 수 없다. 지지된 자세에서 멈출지, 정해 둔 자세로 이동할지, 언제 출력을 해제할지는 몸체와 부하를 보고 검증한다.

이 장에서는 그 회로와 제어 코드를 완성하지 않는다. 대신 임무 카드에 **정지 정책을 반드시 결정할 항목**으로 남긴다. 실제 배선은 5장에서 전원 조건을 확인한 뒤 진행하고, 관절의 범위와 정지 동작은 이후 보정·통합 과정에서 시험한다.

연결 복구와 동작 재개는 다르다

PC와 로봇의 연결이 끊겼다가 돌아왔다고 생각해 보자. 통신은 복구되었지만 사용자는 로봇을 다른 곳으로 옮겼을 수 있다. 그 상태에서 끊기기 전의 전진 명령을 자동으로 다시 실행하면, 현재 상황과 맞지 않는 행동이 된다.

그래서 첫 설계에서는 연결 복구 후 새 요청을 기다리도록 정한다. “연결됨”, “동작 가능”, “실행 중”을 같은 상태로 합치지 않는 것이다. 이 작은 구분이 나중에 재접속 로직과 호스트 화면의 기준이 된다.

1.8 첫 실습: 임무 카드와 시스템 스케치 만들기

이 실습은 약 40~60분을 예상한 설계 활동이다. 시간은 진행 예시이며 정해진 속도로 끝내야 하는 시험은 아니다. 준비물은 종이·펜 또는 문서 편집기, 그리고 만들 로봇에 대한 아이디어다. 보드나 ROS 2 설치는 필요하지 않다.

1단계. 경로를 고르고 한 문장으로 적는다

고정형 로봇과 이족보행형 가운데 먼저 완성할 경로를 선택한다. 모양만 보고 고르기보다 관찰하고 싶은 행동을 기준으로 생각하자. 사람에게 반응하는 팔과 표시 장치에 관심이 있다면 고정형 로봇, 관절 자세가 걸음으로 이어지는 과정에 관심이 있다면 이족보행형으로 시작할 수 있다.

다음 양식을 복사해 문서나 노트에 작성한다. robot-mission.md라는 이름으로 저장해도 좋다. 실행 프로그램이 아니라 학습 기록 파일이다. 이 샘플에서는 아래 본문 양식을 그대로 복사해 사용한다.

프로젝트 이름:
 선택 경로: 고정형 로봇 / 이족보행형
 사용자와 사용 장소:

한 문장 임무:
 [누가] [어떤 상황에서] [어떤 입력을 주면]
 로봇이 [무엇을 하고] [어떤 상태로 끝나는가]?

필수 기능 3개:
 이번에 제외할 기능:
 로컬 정지와 통신 중단 때의 행동:
 관찰할 결과:
 아직 모르는 것과 확인할 장:

“이것도 있으면 좋겠다”는 기능은 별도 후보 목록으로 옮긴다. 모든 후보를 필수 기능에 넣으면 첫 로봇의 크기가 계속 커진다. 필수 기능 세 가지는 정상 행동, 원격 연결, 정지·복구를 고려해 고른다.

2단계. 부품 이름보다 역할을 먼저 그린다

종이에 PC, Agent, Pico 2 W, 입력, 출력 장치, 전원, 몸체를 표시한다. 먼저 아래와 같이 역할을 연결해 보자.

사람 → 터치 입력 → Pico의 판단 → 관절·빛·소리

PC의 행동 요청 → Agent → Pico의 판단

Pico의 입력·동작 상태 → Agent → PC의 표시

전원 → Pico와 출력 장치

몸체 → 관절 범위·부품 배치·배선 경로의 조건

이는 전기 배선도가 아니다. 전원 화살표만 보고 핀을 연결하거나, 같은 전압을 모든 부품에 공급하면 안 된다. 전기적 연결과 정격은 5장에서 따로 설계한다. 지금은 정보와 에너지가 어디로 흐르는지 역할 수준에서 표현한다.

정보 화살표에는 “데이터” 대신 “인사 요청”, “터치 상태”, “정지됨”처럼 구체적인 내용을 적는다. 가능하면 요청 방향과 상태 보고 방향을 따로 그린다. 한 선에 모든 의미를 넣으면 무엇이 도착하지 않았는지 구분하기 어렵다.

3단계. 세 가지 시나리오를 만든다

시험 상황	줄 입력 또는 변화	기대할 결과
정상 동작	대기 상태에서 시작 요청	정한 행동 후 대기 상태로 복귀
로컬 정지	동작 도중 정지 입력	경로별 정지 정책 실행·정지 상태 표시
통신 중단	원격 동작 중 새 명령/통신이 끊김	정한 만료 정책 실행·복구 뒤 새 요청 대기

시나리오마다 관찰할 위치도 적는다. 예를 들어 “PC 로그에 요청이 보였다”는 사실만으로 팔 동작을 확인할 수는 없다. 요청 로그, 보드의 수신·실행 상태, 실제 팔의 움직임을 나누어 생각해야 한다.

4단계. 다른 사람이 설계를 읽어 보게 한다

함께 공부하는 사람이 있다면 자신의 설명 없이 임무 카드만 보여 준다. 그 사람이 “무엇을 누르면 어떤 동작을 하고, 언제 멈추는가?”에 답할 수 있는지 확인한다. 혼자 공부한다면 잠시 뒤 다시 읽으며 같은 질문에 답해 보자.

답이 둘 이상으로 갈리는 부분을 고치면 된다. “버튼을 누르면 동작한다” 대신 “대기 상태에서 시작 입력이 새로 들어오면 한 번 실행한다”처럼 조건을 구체화할 수 있다. 구현을 시작하기 전에 발견한 모호함은 수정하기가 쉽다.

예상 결과: 선택 경로, 필수 기능 세 가지, 정보 흐름, 정상·정지·단절 시나리오가 담긴 한두 쪽의 문서가 남는다. 정격 전원과 응답 시간처럼 아직 모르는 항목은 확인할 위치와 함께 표시한다.

1.9 설계가 막힐 때 확인할 것

만들 것이 너무 많아 보인다. 최종 로봇에서 가장 짧게 보여 줄 장면 하나를 고르자. “터치하면 한 번 인사한다” 또는 “시작 요청 뒤 한 걸음 이동하고 멈춘다”로 줄이면 필요한 입력과 출력이 보인다. 카메라·앱·음성은 후보 목록으로 옮길 수 있다.

PC와 Pico 중 어디에 코드를 넣어야 할지 모르겠다. 그 기능이 PC가 꺼져도 필요할지 질문해 보자. 관절 갱신과 로컬 정지처럼 몸체 가까이에서 계속 확인해야 하는 기능은 Pico 쪽 책임으로 두고, 사용자가 행동을 선택하거나 로그를 보는 기능은 PC 쪽에서 시작해 본다. 이후 통합 시험에서 경계를 조정할 수 있다.

무선으로 제어하니 로컬 입력이 필요 없다고 생각했다. 연결 문제를 겪는 동안에도 로봇 상태를 바꾸거나 중단할 방법을 생각해야 한다. 첫 스케치에 로컬 입력을 표시하고, 통신 중단과 정지 요청의 처리 위치를 다시 확인하자.

성공 기준을 숫자로 쓰기 어렵다. 아직 측정할 수 없다면 관찰 순서와 측정 방법부터 적는다. “안정적으로 걷는다”보다 “같은 바닥에서 세 번 전진을 시도하고 거리·시간·넘어짐·정지 상태를 기록한다”가 구체적이다. 최종 합격값은 시험 전에 정하고, 실패한 뒤 결과에 맞춰 바꾸지 않는다.

1.10 이 책에서는 작은 성공을 누적한다

각 장은 로봇을 새로 시작하는 실습이 아니라 이전 결과에 한 가지 책임을 추가하는 과정이다. 처음에는 관절 한 개를 확인하고, 다음에는 입력에 반응하게 하며, 이후에는 여러 작업과 원격 요청을 함께 처리한다.

임무 카드에서 재현 가능한 로봇까지

각 단계는 학습 목표이며, 현재 펌웨어와 실물 시험의 완료를 뜻하지 않는다

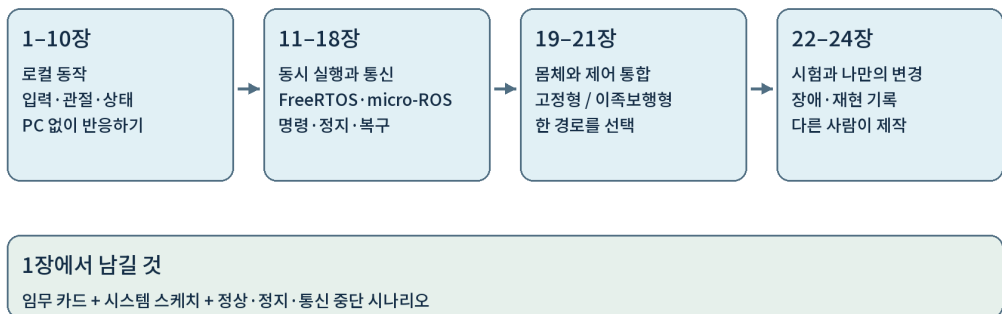


그림 1-4. 로컬 반응에서 동시 실행과 원격 연결을 거쳐 몸체 제작·검증으로 진행한다. 한 제작 경로를 완성한 뒤 자신의 요구를 추가한다.

10장까지는 로컬 로봇을 만든다. 개발 환경과 C, 빌드·업로드를 준비하고 전원·서보·터치·표시 장치를 순서대로 확인한다. 통신 없이도 입력과 행동이 이어지는 상태기계를 만든다.

18장까지는 연결된 제어부를 만든다. FreeRTOS의 태스크와 큐로 책임을 나누고, 직렬 micro-ROS 왕복을 확인한 뒤 Wi-Fi로 확장한다. 메시지 메모리, 잘못된 요청, 연결 복구, 행동 시작과 취소를 다룬다.

19-21장에서는 몸체를 설계하고 조립한다. 실제 부품을 기준으로 BOM과 배선·치수·조립 순서를 정한다. 20장 고정형 로봇 또는 21장 이족보행형 가운데 선택한 경로를 제작한다. 혼을 조립한 뒤에는 관절 방향과 범위를 다시 확인한다.

22-24장에서는 검증하고 다시 설계한다. 정상 동작만 아니라 통신 단절과 명령 집중도 시험한다. 자신의 요구를 한 가지 반영하고, 다른 사람이 다시 만들 수 있도록 문서와 결과를 정리한다.

실물 부품이 아직 없어도 이 장의 임무와 구조 설계는 할 수 있다. 다만 문서와 화면에서 확인한 결과로 전원·관절 간섭·보행을 검증했다고 말할 수는 없다. 하드웨어가 필요한 단계에서는 실제 부품으로 다시 확인한다. 소프트웨어 구조를 검증한 것과 물리적인 로봇을 완성한 것을 구분하는 습관을 처음부터 갖자.

AI와 함께 문제 해결하기

이 장에서는 AI를 임무 카드의 첫 독자로 삼는다. 먼저 자신의 생각을 쓰고, AI가 던지는 질문에 답하면서 입력·행동·종료 상태를 구체화한다. 완성 문장을 대신 받아 적기보다 어떤 조건을 새로 정했는지 남기자.

문제 1. 막연한 아이디어를 시험 가능한 임무로 바꾸기

“사람에게 반응하는 로봇”을 만들고 싶다. 사용할 상황과 입력을 스스로 정한 뒤, AI와 대화하며 성공과 중단을 관찰할 수 있는 임무 카드로 고쳐 보자.

함께 제공할 자료: 자신의 임무 카드 초안, 선택한 제작 경로, 아직 정하지 않은 조건.

예시 프롬프트

Pico 2 W 로봇의 임무를 구체화하려고 한다.
 선택 경로와 임무 초안: [내 초안]
 사용자·입력·행동·종료 상태 중 모호한 표현을 찾아 줘.
 완성안을 먼저 쓰지 말고 내가 결정할 질문을 한 번에 하나씩 해 줘.
 내 답변 뒤에 정상·중단 상황에서 관찰할 항목을 제안해 줘.
 미정인 핀·전압·응답 시간을 임의로 채우지 마.

직접 확인하고 남길 것: 대화 전후 임무 카드와 수정 이유를 비교한다. 다른 사람이 카드만 보고 성공·중단 여부를 판단할 수 있는지 확인한다.

문제 2. 시스템 스케치의 책임을 설명하기

PC·Agent·Pico·FreeRTOS·입출력 장치를 연결한 스케치를 그리고, AI가 설명한 역할과 자신의 설명을 대조해 보자. 같은 프로그램 구조로 가능한 것과 몸체에 따라 다시 설계할 것을 구분한다.

함께 제공할 자료: 스케치 또는 화살표를 풀어 쓴 문장, 각 구성 요소에 대한 자신의 설명.

예시 프롬프트

이 장 원고와 내 시스템 설명을 제공한다: [자료]
내가 PC·Agent·Pico·태스크·코어에 부여한 책임을 검토해 줘.
서로 혼동한 개념이 있으면 원고의 근거와 확인 질문을 제시해 줘.
고정형에서 이족보행형으로 바꿀 때 유지·재검토할 항목을 물어봐 줘.
아직 배우지 않은 구현을 완성된 기능처럼 가정하지 마.
내가 고친 스케치를 보낸 뒤 다시 검토해 줘.

직접 확인하고 남길 것: 모든 화살표에 요청 또는 관측 데이터의 이름을 붙인다. AI가 지정한 항목 가운데 채택한 것과 근거를 찾지 못해 보류한 것을 기록한다.

문제 3. 명령 로그만으로 성공을 말할 수 있는지 토론하기

가상의 기록에는 “각도 명령 전송 완료”와 “평균 처리 시간 2 ms”만 있다. 목표는 팔 동작 완료와 20 ms 마감 준수다. 이 기록으로 주장할 수 있는 범위를 AI와 논의하고 다음 관찰을 설계해 보자.

함께 제공할 자료: 목표와 가상 기록, 자신이 먼저 쓴 성공 판정과 이유.

예시 프롬프트

다음은 실제 실험이 아닌 검토용 상황이다.
목표는 팔 동작 완료와 20 ms 마감 준수다.
기록은 명령 전송 완료, 평균 2 ms, 일부 처리 80 ms다.
내 판정과 이유: [내 설명]
내 주장에 필요한 증거가 빠졌는지 질문해 줘.
장비 없이 할 분석과 실물에서 할 관찰을 나누어 제안해 줘.
통신 복구 뒤 동작 재개 조건도 내게 확인해 줘.

직접 확인하고 남길 것: 관찰한 사실·추정·추가로 필요한 측정을 나눈다. AI의 설명을 실험 결과로 쓰지 않고 자신의 임무 카드에 확인 방법을 반영한다.

핵심 정리

- 작은 로봇도 입력·판단·동작·상태 확인과 전원·몸체가 연결된 시스템이다.
- PC는 행동 선택과 표시를, Pico는 장치 제어와 로컬 동작을 맡도록 책임을 나누어 설계한다.

- micro-ROS Agent는 MCU의 micro-ROS 통신을 ROS 2 시스템과 연결한다.
- FreeRTOS 태스크, ROS 2 Node, CPU 코어는 서로 다른 개념이다.
- 임베디드 시스템은 장치 안에서 특정 기능을 맡는다. micro-ROS는 장치 제어 펌웨어를 ROS 2의 데이터·도구와 연결한다.
- 반응 속도와 마감 시간 준수는 따로 확인한다. 로컬 제어와 FreeRTOS를 사용해도 시간 요구의 설계·분석·측정이 필요하다.
- 고정형 로봇은 상호작용과 표현, 이족보행형은 관절 보정과 보행에 초점을 두며 공통 제어 기반을 공유한다.
- 임무는 사용자·입력·행동·종료 상태로 구체화하고, 정지와 통신 중단도 함께 설계한다.
- 첫 산출물은 임무 카드와 시스템 스케치다. 성능값과 실물 결과는 측정·검증 전까지 확정된 사실로 쓰지 않는다.

1장 마무리

실습 완료 체크리스트

- ☐ 사용자·상황·입력·기대 행동·종료 상태가 들어 있는 임무를 썼다.
- ☐ 고정형 로봇 또는 이족보행형 경로를 선택하고 이유를 설명했다.
- ☐ 필수 기능 세 가지와 이번에 제외할 기능을 분리했다.
- ☐ PC·Agent·Pico·입력·출력·전원·몸체를 스케치에 표시했다.
- ☐ 정보의 화살표에 요청과 상태의 구체적인 내용을 적었다.
- ☐ 정상·로컬 정지·통신 중단 시나리오와 관찰 위치를 정했다.
- ☐ 미정인 전원·보정·응답 시간에는 확인할 단계와 근거를 적었다.
- ☐ 다른 사람 또는 미래의 자신이 문서만 보고 동작을 설명할 수 있다.

참고 자료

- Raspberry Pi 공식 문서: Pico 계열 보드
<https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>
- ROS 2 공식 문서 원본: Node의 역할
https://github.com/ros2/ros2_documentation/blob/jazzy/source/Concepts/Basic/About-Nodes.rst
- micro-ROS 공식 문서: 기능과 구조
<https://micro.vulcanexus.org/docs/overview/features/>
- micro-ROS 공식 문서: RTOS 구성
https://github.com/micro-ROS/micro-ROS.github.io/blob/master/_docs/overview/rtos/index.md

- FreeRTOS 공식 소개

<https://www.freertos.org/Why-FreeRTOS/What-is-FreeRTOS>

- 고정형 로봇 프로젝트 기록과 소스 안내

<https://freshmea.github.io/madam/pages/bindbot-2026.html>

- 고정형 로봇 기준 소스 스냅샷

https://github.com/freshmea/micro_ros_pico_dev/tree/393f96b6e484b5c70245b821dddd3aca2d7cc1fc

- 참고 이족보행 로봇 제작 안내

<https://www.ottodiy.com/blog/ottodiykit>

- 참고 보행 라이브러리의 Arduino 인터페이스

<https://github.com/OttoDIY/OttoDIYLib/blob/main/src/Otto.h>

다음 장 예고

다음 장에서는 PC의 개발 환경과 Pico 2 W를 연결한다. 프로그램을 작성하는 장소, 빌드하는 장소, 펌웨어가 실행되는 장소를 구분하고 도구와 버전을 기록한다. 이 장에서 적은 임무 카드를 옆에 두고, 그 로봇을 만들기 위한 첫 환경 확인표를 작성해 보자.

02

CHAPTER

Pico 2 W 개발 환경과 재현 가능한 버전



학습 목표

- Pico 2 W의 사양과 대안 보드의 선택 기준을 설명한다.
- SDK·컴파일러·RTOS의 역할과 버전을 구분한다.

이 장의 학습 흐름

STEP 01
보드 선택

STEP 02
SDK 이해

STEP 03
환경 기록

완료 기준

- 공식 자료로 후보 보드와 총비용을 비교한다.
- 검증 완료와 후보 버전을 구분한 환경표를 만든다.

이번 장의 결과물 보드 선택 근거표와 환경 확인표

로봇을 만드는 데 사용할 수 있는 보드는 Pico 2 W 하나뿐이 아니다. 이번 장에서는 여러 후보를 비교해 기준 보드의 선택 이유를 이해하고, 그 선택을 재현 가능한 개발 환경으로 연결한다.

같은 소스를 받아도 한 사람은 빌드에 성공하고 다른 사람은 헤더를 찾지 못한다. 프로그램을 만드는 도구와 의존성이 다르기 때문이다. 환경 준비는 설치 목록을 채우는 일이 아니라 **어떤 조합으로 어떤 결과를 만들었는지 기록하는 일**이다. 1장의 시스템 스케치 옆에 환경 확인표를 한 장 더 놓자.

학습 목표와 완료 기준

Pico 2 W의 선택 이유와 공식 사양을 설명하고, 다른 보드 두 개 이상을 자신의 요구로 비교한다. Pico SDK·컴파일러·RTOS의 역할을 구분하고 버전과 경로를 기록한다. 완료 결과는 보드 선택 근거표, 환경 확인표와 의존성 목록이다. SDK 최소 예제, RTOS 예제, micro-ROS 왕복을 순서대로 통과한 조합만 최종 기준 환경이 된다. 이 책의 통합 버전은 아직 실물 재현으로 확정하지 않았다.

개발 환경을 하나의 생산 과정으로 이해하기

독자가 처음 만나는 어려움은 설치해야 할 프로그램이 많다는 사실보다 서로 다른 프로그램이 비슷한 오류를 낸다는 사실이다. 헤더를 찾지 못하는 오류, 실행 파일을 찾지 못하는 오류, 보드를 찾지 못하는 오류가 모두 “설치 문제”처럼 보인다. 그러나 세 문제는 발생한 단계가 다르다. 이번 장에서는 설치 명령을 실행하기 전에 소스가 보드의 동작이 되는 경로를 그린다.

편집기에서 작성한 C 소스는 텍스트다. 컴파일러는 이 텍스트를 특정 CPU가 이해하는 명령으로 바꾼다. PC의 x86-64 명령과 Pico 2 W의 Arm 명령은 서로 바꿔 실행할 수 없다. PC에서 실행되는 컴파일러가 다른 CPU용 결과물을 만드는 것을 **크로스 컴파일**이라고 한다. 컴파일러 자체의 실행 장소는 PC이고 결과물의 실행 장소는 보드라는 두 사실을 구분해야 한다.

SDK는 GPIO·타이머·USB 등 보드 기능을 사용하는 코드와 빌드 설정을 제공한다. FreeRTOS는 태스크를 실행하고 기다리는 규칙을 제공한다. micro-ROS 라이브러리는 MCU의 노드와 메시지를 Agent에 연결한다. 이 계층들은 서로의 일을 대신하지 않는다. SDK 예제만으로 ROS 토픽이 나타나지 않고, ROS 2를 설치해도 Arm 컴파일러가 자동으로 준비되는 것은 아니다.

총	독자가 던질 질문	확인 자료
PC 운영체제	이 명령은 Windows인가 Linux인가?	터미널 종류, OS 정보
빌드 도구	어떤 CPU용으로 만들고 있는가?	컴파일러 경로, CMake 출력
SDK와 보드	어떤 핀과 무선 칩을 전제로 하는가?	보드 정의, SDK 커밋
실행 기반	한 루프인가, RTOS 태스크인가?	초기화와 스케줄러 시작 코드

층	독자가 던질 질문	확인 자료
통신	직렬 바이트인가, UDP 데이터그램인가?	전송 구현, Agent 실행 옵션
로봇 동작	받은 값을 누가 검증하고 적용하는가?	제어 상태기계, 보정표

개발 환경 기록은 이 표의 각 행을 채우는 과정이다. 설치 명령의 종료 코드가 0이라는 기록만으로는 부족하다. 그 명령이 어느 계층을 확인했는지 함께 적어야 다음 오류를 좁힐 수 있다.

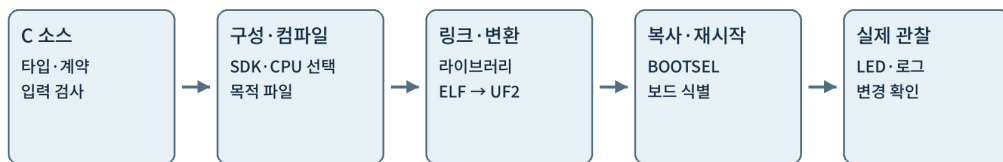
이 책의 두 가지 실행 경로

이 책은 PC에서 실행하는 제어 논리 실습과 Pico에서 실행하는 펌웨어 실습을 함께 사용한다. PC 실습에서는 실제 서보 대신 숫자 배열을 출력한다. 입력 검사나 정지 후 재시작 규칙은 숫자만으로도 잘못된 구현을 찾을 수 있기 때문이다. 반면 전압·펄스·스케줄링·USB 열거는 실물에서 확인한다. 두 경로를 구분하면 하드웨어가 아직 없더라도 학습을 진행하면서, 보드를 받았을 때 무엇을 추가로 검증해야 하는지 알 수 있다.

이 샘플 3장의 전체 C 코드는 PC 논리 실습이다. 4장에서 다루는 최소 펌웨어는 SDK와 보드를 연결하는 다음 단계이며 이 샘플에는 포함하지 않았다. 기존 개발 저장소의 bindbot은 더 많은 의존성을 가진 통합 사례다. 이 세 대상을 한 번에 빌드하려 하지 말고, 적은 의존성을 가진 것부터 확인한다.

소스에서 보드 동작까지, 확인 경계는 다섯 개

개념도 · 각 단계의 성공은 다음 단계의 성공을 대신하지 않는다



PC 논리 실습은 별도 경로

같은 C의 입력·시간·상태를 검사한다. GPIO·USB·PWM·기구 반응은 실물에서 확인한다.

그림 2-1. 소스·빌드·업로드·관찰의 검증 경계.

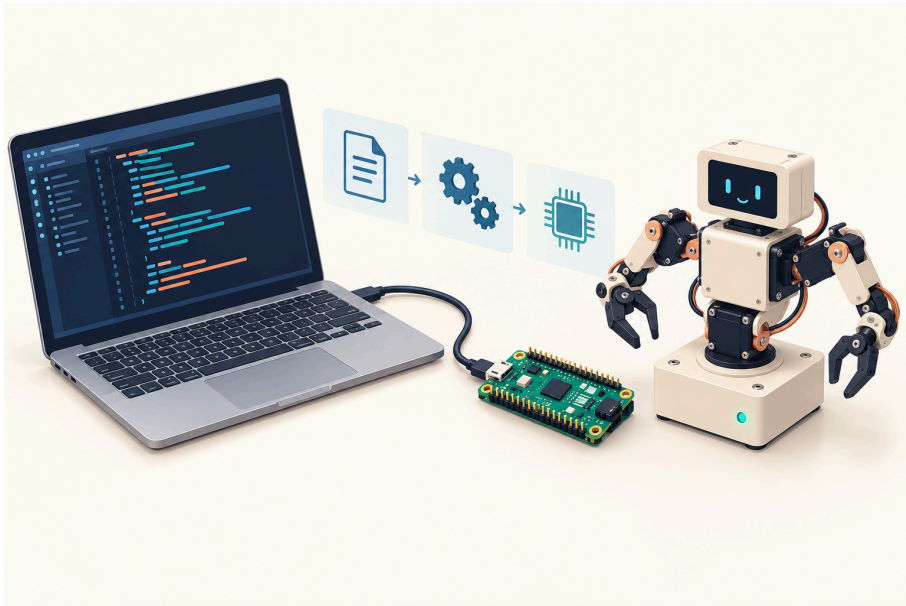


그림 2-2. 소스 작성·펌웨어 생성·보드 실행의 서로 다른 작업 장소. 생성형 개념 삽화이며 특정 보드 배선이나 완성 로봇 사진이 아니다.

2.1 왜 이 책에서는 Pico 2 W를 선택했는가

이 책의 목표는 Pico라는 제품의 사용법을 외우는 데 있지 않다. 센서 입력을 해석하고, 동작을 시간에 따라 갱신하고, 통신이 끊겨도 로컬에서 정지하며, 결과를 ROS 2와 주고받는 구조를 배우는 것이 목표다. 이 구조는 여러 마이크로컨트롤러에 적용할 수 있다. **Pico 2 W는 설명과 실습 조건을 맞추기 위해 선택한 기준 보드다.** 첫 실습의 변수를 줄인 뒤, 23장에서는 자신의 요구에 맞게 다른 보드로 옮기는 설계도 할 수 있다.

기준 보드를 정할 때는 최고 클럭 하나로 순위를 매기지 않았다. 이 책에서 누적할 실습과 기존 참고 코드가 연결되는지를 함께 보았다.

선택 이유	이 책의 학습과 연결되는 지점
비교적 낮은 보드 가격	제조사 발표 기준 미화 7달러인 기본형으로 시작한다. 센서·전원·기구 부품에도 예산을 나눌 수 있다.
로컬 제어와 무선 연결을 한 보드에서 경험	먼저 GPIO·PWM·I2C를 다루고, 직렬 통신을 거쳐 Wi-Fi와 UDP로 확장한다.
제한을 관찰할 수 있는 메모리와 주변장치	버퍼·태스크 스택·큐를 계산하고 PIO가 처리할 일을 구분하는 실습에 연결한다.
C/C++ SDK와 공개 문서	보드 정의, 라이브러리 연결, 빌드 결과를 소스 수준에서 따라갈 수 있다.

선택 이유	이 책의 학습과 연결되는 지점
BOOTSEL과 SWD	파일 복사 방식으로 처음 펌웨어를 올리고, 필요할 때 디버거를 사용하는 단계로 확장한다.
책의 기준 개발 코드와 연결	참고 저장소가 pico2_w와 RP2350 Arm 포트를 사용하므로 실제 코드의 구조를 분석할 수 있다.

가격 근거는 Raspberry Pi의 **Pico 2 W 출시 발표**다. 국내 배송비와 헤더 납땜 비용까지 포함한 구매 가격이라는 뜻은 아니다. 기준 코드의 연결은 이 책의 편집상 선택 이유이며, 보드 제조사가 책의 통합 펌웨어를 검증했다는 뜻도 아니다.

다른 목적에는 다른 선택이 자연스럽다. Wi-Fi가 필요 없는 고정 장치라면 Pico 2를 검토할 수 있다. 아주 작은 센서 장치라면 XIAO 계열의 크기가 유리할 수 있다. 중단점을 걸고 주변장치 레지스터를 관찰하는 수업이라면 디버거를 갖춘 Nucleo 보드가 편리하다. 영상 인식이나 큰 AI 모델 실행이 목적이라면 MCU의 클럭만 높이기보다 Linux 호스트가 계산하고 MCU가 장치를 제어하도록 역할을 나누는 편이 적합할 수 있다.

이 시리즈에서 말하는 **AI를 활용한 학습**은 AI와 요구·코드·오류·측정 결과를 검토하는 학습 방법이다. 반드시 보드 안에서 생성형 AI 모델을 실행해야 한다는 뜻은 아니다. AI에게 보드 이름 하나를 추천받는 데서 끝내지 말고, 추천의 전제와 공식 자료를 함께 확인해 보자.

2.2 Pico 2 W의 사양을 로봇 설계의 언어로 읽는다

먼저 세 이름을 구분한다. **RP2350**은 마이크로컨트롤러 칩이고, **Pico 2 W**는 그 칩에 플래시 메모리·전원 회로·USB·무선 회로 등을 연결한 개발 보드다. **Pico SDK**는 이 하드웨어를 C/C++에서 사용하는 소프트웨어 개발 도구 모음이다. 칩이 제공하는 기능과 보드에서 외부 핀으로 꺼내 쓸 수 있는 기능이 항상 같지는 않다.

다음 표는 **Pico 2 공식 제품 사양**과 **Pico 2 W 데이터시트**의 수치를 정리한 것이다. 2026년 9월 30일 확인한 문서를 기준으로 하며, 제품군 전체의 최대 사양을 이 보드의 사양으로 섞지 않는다.

연산과 메모리

항목	Pico 2 W의 사양	설계에서 읽을 의미
MCU	Raspberry Pi RP2350	핀 이름만이 아니라 CPU·주변장치·SDK 포트를 함께 확인한다.
CPU	최대 150MHz, 듀얼 Cortex-M33 또는 듀얼 Hazard3 RISC-V	이 책은 Arm 경로를 사용한다. 두 종류를 더해 4코어가 동시에 실행되는 보드로 계산하지 않는다.

항목	Pico 2 W의 사양	설계에서 읽을 의미
SRAM	520KB	실행 중 데이터·스택·힙·통신 버퍼 등이 사용한다. 전부를 애플리케이션 배열에 할당할 수 있는 것은 아니다.
플래시	보드에 4MB QSPI 플래시	펌웨어 저장 공간이다. RAM 부족을 같은 용량의 플래시로 바로 대체할 수 없다.

150MHz는 초당 실행할 수 있는 제어 루프 횟수가 아니다. 한 번의 루프에는 계산, 장치 접근, 대기, 인터럽트 처리가 들어간다. 또 두 코어가 있다고 모든 작업이 저절로 두 배 빨라지지 않는다. 같은 I2C 버스나 상태 구조체를 공유하면 조정이 필요하다. 이 책은 단일 코어에서 책임을 명확히 한 뒤, 12장에서 공유 자원과 코어 배치를 검토한다.

메모리는 저장 공간과 작업 공간을 나누어 생각한다. 동작 시나리오 상수는 플래시에 둘 수 있지만 수신 중인 메시지와 태스크 스택은 실행 중 사용할 공간이 필요하다. 16장에서 만드는 메모리 예산 표에는 프로그램이 실제로 사용하는 영역과 남겨 둘 여유를 적는다. 사양표의 SRAM 숫자만 보고 사용할 토픽 수를 확정하지 않는다.

핀과 주변장치

항목	보드에서 확인할 사양	실습과 연결
외부 GPIO	다기능 GPIO 26개, 보드 I/O 기준 3.3V	터치 입력·서보 제어 신호·버저·표시 장치에 배정한다.
ADC	외부 아날로그 입력 3개, ADC 분해능 12비트	디지털 핀 전체가 아날로그 입력인 것은 아니다. 기준 전압과 잡음도 측정값에 영향을 준다.
직렬 주변장치	UART 2개, SPI 2개, I2C 2개	장치 수와 버스 수를 구분하고 핀 기능의 겹침을 확인한다.
PWM	제품 사양 기준 16채널	GPIO 수와 독립 타이머 수는 다르다. 주기를 공유하는 채널을 확인한다.
PIO	3블록, 상태기계 총 12개	일정한 타이밍의 입출력을 전담시킬 수 있다. 9장의 네오픽셀 실습에 연결한다.

항목	보드에서 확인할 사양	실습과 연결
USB	USB 1.1, 장치 또는 호스트 기능	책에서는 업로드와 통신을 중심으로 쓴다. 커넥터가 있다고 모든 USB 기능이 동시에 제공되지는 않는다.
디버그	3핀 SWD 접점	중단점·실행 상태 관찰에는 대응하는 외부 디버거와 연결이 필요하다.

GPIO는 여러 기능 중 무엇을 그 핀에 연결할지 선택하는 접점이다. 예를 들어 I2C에 배정한 핀을 같은 순간에 독립적인 PWM 출력으로도 쓰겠다고 계산하면 안 된다. 두 I2C 장치가 주소 충돌 없이 같은 버스에 연결될 수 있다면 핀 두 개를 함께 사용할 수 있다. 반대로 같은 주소의 장치를 분리해야 한다면 다른 버스나 주소 변경, 멀티플렉서가 필요할 수 있다. 보드 선택표에는 GPIO 총수뿐 아니라 실제 핀 배정표가 따라와야 한다.

보드 가장자리의 물리적인 핀 번호와 GP로 시작하는 GPIO 번호도 구분한다. 같은 숫자라고 같은 접점이 아니다. 배선 전에는 데이터시트의 핀 그림에서 전원·GND·신호를 확인하고, 코드에는 선택한 GPIO 번호를 기록한다. Pico 2 W의 사용자 LED는 무선 칩 쪽에 연결되어 있다. 4장 예제가 일반 GPIO를 직접 켜는 대신 `cyw43_arch_gpio_put()`을 사용하는 이유다. 무선 없는 Pico의 LED 코드를 그대로 복사하면 이 차이를 놓칠 수 있다.

무선·전원·크기

항목	Pico 2 W의 사양	조립 전에 확인할 점
무선 칩	Infineon CYW43439	RP2350과 연결된 별도 무선 회로다. SDK의 대응 드라이버를 사용한다.
Wi-Fi	단일 대역 2.4GHz, 802.11n	5GHz 전용 네트워크에 연결하는 보드로 가정하지 않는다.
Bluetooth	Bluetooth 5.2	하드웨어 기능과 책에서 구현하는 통신은 구분한다. 본문의 micro-ROS 무선 경로는 Wi-Fi/UDP다.
주 전원 입력	VSYS 1.8~5.5V	이 범위는 GPIO에 넣어도 되는 신호 전압이 아니다.
USB 단자	Micro USB B, 전원·데이터	충전 전용 케이블과 데이터 케이블을 구분한다.

항목	Pico 2 W의 사양	조립 전에 확인할 점
기판 크기	21×51mm, 40핀 형태	헤더 높이·케이블 굵힘·안테나 주변 여유까지 몸체에 반영한다.

전원은 이름을 보고 구분하자. VBUS는 USB에서 들어오는 전원 경로이고 VSYS는 보드의 시스템 전원 입력이다. 3V3는 보드의 3.3V 전원이며, GPxx는 장치에 신호를 전달하는 핀이다. 서보의 동작 전류를 신호 핀이나 보드의 3.3V 출력으로 공급하는 방식으로 설계하지 않는다. 부품에 맞는 구동 전원과 공통 기준 전위는 5장에서 검토한다. 외부 5V 신호를 연결할 때도 입력 전원 범위를 근거로 허용된다고 판단하지 말고, 해당 핀의 전기 조건과 레벨 변환 필요성을 확인한다.

무선 안테나 주위에 금속 부품·배터리·배선을 밀착하면 통신 조건이 달라질 수 있다. 데이터시트의 안테나 주변 배치 지침을 먼저 보고 19장에서 몸체 내부 공간을 확보한다. 벤치에서 성공한 Wi-Fi 연결을 조립 후에도 다시 시험하는 이유다. 전력 소모 역시 무선 송수신·대기·연결 재시도·주변장치 동작에 따라 달라지므로, 보드 하나의 대표 전류를 로봇 전체 배터리 계산에 그대로 넣지 않는다.

공식 문서를 읽는 순서

제품 소개는 후보를 줄이는 데, 보드 데이터시트는 핀·전원·기구를 확인하는 데 쓴다. 더 깊은 타이머·메모리·CPU 동작은 RP2350 데이터시트에서 찾는다. 소프트웨어 함수 사용법은 SDK 문서로 넘어간다. 문서에서 수치가 다르면 제품 모델, 칩 패키지, 문서 판본, 적용 조건부터 비교하고 AI에게 서로 다른 항목을 합쳐 달라고 하지 않는다. 학습 기록에는 문서 제목·URL·확인일과 실제 보드 판본을 함께 남긴다.

2.3 Pico SDK는 무엇을 제공하는가

SDK는 Software Development Kit의 줄임말이다. Raspberry Pi의 C/C++ SDK 안내는 설치 방법과 개발 흐름을 설명하고, pico-sdk 저장소는 라이브러리와 빌드 지원을 제공한다. pico-examples는 기능별로 작게 확인할 수 있는 공식 예제다.

SDK를 처음부터 끝까지 외울 필요는 없다. 현재 연결하려는 장치가 무엇인지 정하고, 대응하는 최소 예제를 찾은 뒤, 필요한 라이브러리와 초기화 순서를 읽으면 된다. 예제의 성공 조건을 먼저 확인하면 통합 로봇의 오류와 장치 자체의 오류를 구분하기 쉬워진다.

SDK에서 만나는 이름	역할	이 책에서 처음 확인할 곳
pico2_w 보드 정의	보드에 맞는 기본 설정을 고른다.	2장의 SDK 경로 확인, 4장의 CMake 구성
pico_stdlib	기본 입출력·시간·일반적인 보드 프로그램에 필요한 기능을 묶는다.	4장의 최소 LED·USB 로그

SDK에서 만나는 이름	역할	이 책에서 처음 확인할 곳
hardware_pwm, hardware_i2c	PWM과 I2C 등 주변장치를 사용하는 API다.	6장의 서보, 8장의 OLED
hardware_pio와 PIO 빌드 지원	PIO 프로그램과 상태기계를 연결한다.	9장의 네오픽셀
pico_cyw43_arch_*	무선 칩과 네트워크 사용 방식에 맞는 통합을 제공한다.	4장의 LED, 15장의 Wi-Fi
pico_add_extra_outputs()	빌드 타겟의 추가 산출물을 생성하도록 설정한다.	4장의 ELF·UF2 확인

위 이름은 SDK 및 책의 실제 코드에서 찾아볼 항목이지 설치해야 할 독립 프로그램 여섯 개가 아니다. pico_cyw43_arch_none으로 무선 칩에 연결된 LED를 제어한 성공이 Wi-Fi 연결 성공을 뜻하지 않는 것도 같은 이유다. 네트워크가 필요한 단계에서는 그에 맞는 구성과 초기화가 추가된다.

Pico SDK와 컴파일러, FreeRTOS, micro-ROS는 역할이 다르다. SDK는 하드웨어 접근과 빌드를 돕고, 컴파일러는 C/C++를 대상 CPU의 명령으로 바꾼다. FreeRTOS는 여러 태스크의 실행과 대기를 관리하며, micro-ROS는 메시지와 통신을 ROS 2 세계에 연결한다. SDK를 설치했다고 세 기능이 모두 완성되는 것은 아니다. MicroPython도 Pico에서 사용할 수 있지만 이 책은 C·메모리·RTOS·통신 경계를 배우기 위해 C/C++ SDK 경로를 택한다.

-DPICO_BOARD=pico2_w는 CMake에 보드 정의를 선택하도록 전달하는 옵션이다. 이것은 다른 보드의 전압이나 배선을 자동 변환하는 스위치가 아니다. 4장의 firmware/ch04-blink를 구성할 때 실제로 사용하며, SDK에는 그 보드 정의 파일이 있어야 한다. PICO_SDK_PATH로 선택한 SDK와 하위 모듈의 커밋을 기록하는 이유도 여기에 있다. SDK 갱신과 보드 변경을 동시에 하지 말고, 하나씩 바꾼 결과를 남긴다.

보드와 펌웨어 언어를 함께 선택한다

Pico SDK의 C/C++ 경로 외에 Rust로 MCU 펌웨어를 작성하는 경로도 있다. 예를 들어 [Embassy의 RP235x](#) 지원을 살펴볼 수 있다. 보드를 고른 뒤에는 언어·HAL·드라이버·실행 기반까지 함께 확인한다. 이 책은 C를 기준으로 실습하며, 3장에서 C와 Rust의 특징과 Rust를 사용하는 Microduck 제품 사례를 소개한다.

언어가 바뀌어도 전원·배선·입출력 요구는 유지된다. 다만 빌드 도구와 드라이버, 태스크 실행 방식, 통신 라이브러리의 연결은 달라질 수 있으므로 SDK나 언어 이름만 바꾸어 같은 펌웨어를 얻을 수 있다고 가정하지 않는다.

2.4 다른 임베디드 보드에는 어떤 선택지가 있는가

다음은 선택의 폭을 보여 주는 비교다. 모든 보드에서 이 책의 펌웨어가 그대로 실행된다는 목록은 아니다. 가격은 **2026년 9월 30일 확인한 미화 기준 참고값**이다. Raspberry Pi 항목은 제조사 공개 기본형 가격이며, 나머지는 명시한 판매처의 단품 표시 가격이다. 세금·배송·환율·국내 유통·헤더·품질 여부는 구매 시 다시 확인한다. 특별 할인거나 대량 구매 단가를 기본 가격으로 섞지 않았다.

Pico 계열: 익숙한 형태에서도 조건이 달라진다

보드·가격 근거	핵심 차이	선택할 상황과 추가 확인
Pico · \$4부터	RP2040, 듀얼 Cortex-M0+ 133MHz, SRAM 264KB, 플래시 2MB, 무선 없음	GPIO·유선 제어 학습. RP2350용 펌웨어·RTOS 포트를 그대로 쓰지 않는다.
Pico W · \$6	RP2040, SRAM 264KB·플래시 2MB, 2.4GHz Wi-Fi	이미 갖고 있다면 로컬·무선 개념 학습에 활용한다. 메모리 예산과 SDK 타깃을 다시 맞춘다.
Pico 2 · \$5부터	RP2350, 150MHz, SRAM 520KB·플래시 4MB, 무선 없음	유선 로봇의 후보. 본문의 Wi-Fi 단계는 별도 통신 수단이 필요하다.
Pico 2 W · \$7	Pico 2 세대의 연산·메모리에 2.4GHz Wi-Fi와 Bluetooth	이 책의 기준. 기본 가격과 헤더가 납땜된 제품 가격을 구분한다.

같은 이름 계열이라도 CPU가 다르면 컴파일 타깃·RTOS 포트·정적 라이브러리가 달라진다. 또한 같은 RP2350을 쓰더라도 무선 칩 유무에 따라 LED 제어와 네트워크 코드가 달라진다. “모양이 비슷하니 UF2를 그대로 복사한다”보다 “재사용할 소스와 다시 빌드할 구성을 나눈다”가 올바른 접근이다.

ESP32·STM32·Teensy 계열: 크기와 개발 경험도 선택 기준이다

보드·단품 표시 가격	주요 자원·특징	선택 시 살필 점
Seeed XIAO ESP32C6 · \$5.20	주 연산 RISC-V 160MHz, SRAM 512KB·플래시 4MB, 2.4GHz Wi-Fi 6·BLE-802.15.4	21×17.8mm 소형. 노출 GPIO와 핀 겸용을 확인하고 C6용 micro-ROS 구성 지원을 따로 조사한다.
Seeed XIAO ESP32S3 · \$7.49	듀얼 Xtensa LX7 240MHz, PSRAM 8MB·플래시 8MB, Wi-Fi·BLE	외부 메모리를 활용할 작업의 후보. 표는 기본형이며 카메라 확장 Sense 모델과 다르다.

보드·단품 표시 가격	주요 자원·특징	선택 시 살필 점
Arduino Nano ESP32, 헤더 포함 · \$19.30	ESP32-S3, 최대 240MHz, SRAM 512KB·PSRAM 8MB·플래시 16MB, USB-C	Arduino 환경에 익숙할 때 검토한다. Nano 핀 표기와 ESP32 GPIO 번호의 대응을 확인한다.
STM32 NUCLEO-F446RE · \$18.82	Cortex-M4 최대 180MHz, SRAM 128KB·플래시 512KB, ST-LINK 디버거 내장	타이머·디버깅 학습의 후보. 무선은 내장하지 않는다. 가격은 아래 DigiKey 단품 기준이다.
Teensy 4.1, DEV-16771 · \$31.50	Cortex-M7 600MHz, RAM 1MB·플래시 8MB, microSD-Ethernet PHY	연산과 유선 입출력이 많은 작업의 후보. Wi-Fi는 내장하지 않으며 Ethernet 커넥터 키트는 별도다.

추가 근거: XIAO ESP32C6 공식 위키, XIAO 계열 제조사 비교, STM32F446RE 사양, NUCLEO-F446RE 단품 가격, Teensy 4.1 기술 자료.

이 표의 PSRAM은 CPU 내부 SRAM과 특성이 같은 메모리로 합산하지 않는다. 접근 지연과 사용할 수 있는 용도, DMA 및 런타임의 제약을 해당 SDK에서 확인해야 한다. 플래시도 부트로더·설정·파일시스템 등에 공간을 나누면 전부를 프로그램에 쓸 수 없다. 서로 다른 CPU의 MHz 숫자만 비교하여 응답 속도의 배수를 계산하는 것도 피한다. 실제 작업과 같은 조건의 측정이 있어야 성능 비교가 된다.

ESP32라는 이름은 단일한 보드 모델을 뜻하지 않는다. C6와 S3는 CPU 구조와 주변장치가 다르고, 같은 S3라도 제작사가 붙인 메모리·커넥터·전원 회로가 다르다. C6의 저전력 보조 코어를 S3의 두 주 연산 코어와 같은 방식으로 계산하지 않는다. Wi-Fi 6나 Thread 같은 무선 기능의 존재도 micro-ROS 전송 구현이 이미 준비되어 있다는 증거는 아니다.

Nucleo는 디버거가 포함되어 있어 칩 가격이 더 싼 보드와 단품 가격만 비교하면 가치를 놓칠 수 있다. Teensy 4.1은 Ethernet PHY가 있는 제품과 없는 변형이 있으므로 정확한 SKU를 확인한다. 어느 쪽이든 보드에 없는 외부 회로를 소프트웨어 설치로 대신할 수 없다.

2.5 보드 가격보다 먼저 요구와 총비용을 정한다

보드를 고를 때 “가장 좋은 보드” 대신 “내 로봇의 필수 조건을 충족하는 보드”를 찾자. 먼저 1장의 임무 카드에서 필수 조건을 뽑는다. 그다음 탈락 조건을 확인하고, 남은 후보끼리 비용·학습 편의·확장성을 비교한다. 필수 핀이 부족한 보드를 낮은 가격 점수로 보상하는 식의 합산은 하지 않는다.

선택 기준	구매 전에 답할 질문	남길 근거
장치와 핀	PWM·I2C·SPI·UART·ADC를 함께 사용할 수 있는가?	부품 목록과 충돌 없는 핀 배정표

선택 기준	구매 전에 답할 질문	남길 근거
전압과 전원	신호 전압, 공급 전류, 모터 전원이 맞는가?	보드·부품 전기 사양과 전원 구성
시간과 메모리	제어 주기, 큐·스택·메시지 공간을 감당하는가?	예상 예산, 추후 실제 측정 계획
통신 경로	USB·UART·Wi-Fi·Ethernet 중 무엇이 필요한가?	Agent까지의 경로와 전송 구현
소프트웨어	해당 모델·CPU·RTOS·ROS 배포판 조합의 근거가 있는가?	공식 지원 표, 예제, 커밋·빌드 기록
조립과 유지	몸체에 들어가며 납땜·수리·재구매가 가능한가?	치수, 커넥터 방향, 공급처와 판본
개발 편의	디버거·로그·오류 복구 경로가 준비되어 있는가?	업로드·디버깅 방법과 추가 장비
총비용	사용 가능한 구성으로 만들면 얼마가 드는가?	보드 외 부품까지 포함한 견적

총비용은 보드 가격에 헤더·케이블·디버거·전원 장치·통신 확장·배송과 세금 등을 더해 계산한다. 이미 가진 장비는 재사용 여부를 표시하고, 모든 후보에 공통인 서보·몸체 비용도 별도 행으로 남긴다. 예를 들어 무선 없는 보드를 고른 뒤 네트워크 확장이 필요해졌다면 확장 부품 값뿐 아니라 핀과 전원, 드라이버 작업도 추가된다. 반대로 유선만 사용하는 로봇에서는 무선 기능이 구매의 필수 조건이 아닐 수 있다.

학습 시간도 현실적인 비용이다. 납땜된 헤더와 잘 정리된 예제, 디버거가 있는 보드는 초기 단가가 높더라도 수업에서 오류를 찾는 시간을 줄일 수 있다. 다만 익숙한 개발 환경이라는 이유만으로 전기 조건이나 통신 요구를 생략해서는 안 된다. 기능을 맞춘 뒤 편의성을 비교한다.

같은 임무로 후보를 비교하는 작은 설계 연습

서보 네 개, I2C OLED 하나, 터치 입력 하나, 네오픽셀 신호 하나, USB 로그와 선택적인 Wi-Fi 제어를 갖는 로봇을 생각해 보자. 장치별 신호선을 단순히 합하면 서보 네 개·I2C 두 개·입력 한 개·LED 한 개로 여덟 개다. 그러나 이것은 아직 완성된 핀 배정이 아니다. 부트 시 상태, PWM 주기 공유, I2C 대체 기능, 보드 내부 사용 핀, 로그 경로와의 충돌을 확인해야 한다.

Pico 2 W에서는 PWM과 PIO를 쓰는 방향으로, ESP32-S3에서는 그 칩이 제공하는 타이머·LED 전송 주변장치를 쓰는 방향으로 드라이버를 설계할 수 있다. Nucleo에서는 유선 통신을 기본으로 잡을지 무선 모듈을 더할지 결정한다. 어느 경우든 임무 카드의 “정지 입력을 우선 처리한다”는 요구는 유지된다. 주변장치 이름이 달라져도 제어 책임까지 바꿀 필요는 없다. 이 사례는 선택 방법을 연습하는 설계 예시이며 해당 보드들의 통합 실행 결과는 아니다.

2.6 다른 보드로 옮길 때 유지할 것과 바꿀 것

보드를 바꾸면 처음부터 모든 개념을 다시 배워야 할 것 같지만, 좋은 설계는 변경되는 부분을 작게 만든다. 각도 범위 검사, 상태 전이, 명령 완료, 토픽의 단위 같은 규칙은 보드 모델보다 로봇의 요구에 가깝다. 반면 PWM 설정과 업로드 방식은 하드웨어에 가깝다.

계층	유지할 수 있는 설계	다시 구현·확인할 부분
임무·상태기계	입력·동작·정지·재시작 조건	바뀐 장치 특성에 따른 조건과 시간
제어 계산	단위, 값 범위, 보간 규칙	자료형 크기·연산 비용·실행 주기
하드웨어 드라이버	읽기·출력·정지 함수의 책임	핀, PWM, I2C, PIO 대체 구현
태스크·시간	장치 소유권, 큐에 넣을 데이터	RTOS 포트, 스택, 타이머, 동기화
ROS 인터페이스	토픽·메시지 타입·단위의 계약	micro-ROS 라이브러리 생성과 전송
빌드·배포	버전 기록과 검증 절차	SDK, 톨체인, 링커 설정, 파일 형식

예를 들어 애플리케이션이 “관절 0을 30도로 이동한다”는 명령을 만들고, 장치 계층이 그 값을 PWM으로 바꾸게 구성하면 보드를 바꾸어도 상위 명령의 뜻은 유지된다. 반대로 모든 통신 콜백에 특정 보드의 레지스터 접근을 흠어 놓으면 이식할 때 수정 범위를 찾기 어렵다. 3장부터 제어 논리를 PC에서 검사하는 이유 중 하나가 이런 분리다.

micro-ROS 지원 여부는 브랜드 이름으로 확정하지 않는다. `micro_ros_arduino`의 보드 지원 표에는 보드별 상태와 보드 패키지 조건이 구분되어 있다. 예를 들어 확인 당시 Teensy 4.1에는 지원 항목이 있지만, 일반 ESP32 항목을 근거로 모든 ESP32-S3-C6 제품에 같은 바이너리를 쓸 수 있다고 확대 해석하면 안 된다. ESP-IDF용 micro-ROS 구성 요소처럼 선택한 SDK에 맞는 경로도 확인한다. 배포판 브랜치와 보드 패키지 버전, 메시지 타입, 전송 방식을 함께 기록해야 한다.

처음 옮길 때는 다음 순서로 작은 성공을 누적한다.

1. 새 보드의 공식 LED·로그 예제를 빌드하고 업로드한다.
2. 모터를 분리한 상태에서 입력과 필요한 버스를 하나씩 확인한다.
3. 제어 논리의 PC 검사를 유지하면서 드라이버를 새 보드에 맞춘다.
4. RTOS를 쓴다면 시간·스택·큐·장치 소유권을 먼저 확인한다.
5. micro-ROS 메시지 한 개를 Agent와 왕복시킨다.
6. 정지·통신 중단·재연결을 시험한 뒤 로봇 전체를 연결한다.

이 순서는 성공을 미리 보장하는 설치 절차가 아니라, 실패 지점을 좁히는 이식 계획이다. 새 보드가 아직 없다면 요구·출처·핀 배정·검증 계획까지 작성하고 실행 칸은 미실시로 둔다. “여러 보드를 쓸 수 있다”는 자신감은 모든 보드가 같다는 생각에서 생기지 않는다. 무엇을 유지하고 무엇을 확인해야 하는지 구분할 때 생긴다.

2.7 호스트와 보드의 경계를 구분한다

에디터는 C 파일을 편집한다. CMake는 소스와 라이브러리의 관계를 정리하고 크로스 컴파일러는 PC에서 RP2350용 기계어를 만든다. 생성한 UF2를 보드에 복사하면 보드가 재시작하여 펌웨어를 실행한다. ROS 2와 Agent는 호스트에서 실행한다.

작업	실행 장소	결과
소스 편집·CMake 구성	Linux 또는 WSL	소스와 빌드 설정
Arm 크로스 컴파일	빌드 환경	ELF·UF2·map
BOOTSEL 복사	보드 볼륨을 보는 OS	보드에 저장된 펌웨어
ROS 2·Agent	호스트	노드·토픽·통신 로그

WSL의 소스와 빌드 폴더는 Linux 홈에 둔다. Windows에서 보드 드라이브가 보인다고 WSL에 직렬 장치도 생기는 것은 아니다. BOOTSEL 복사는 Windows에서 하고 직렬 통신은 USB 전달을 따로 확인할 수 있다. BOOTSEL 저장장치와 펌웨어의 USB CDC도 서로 다른 상태다. 보드 연결은 4장에서 실제 펌웨어를 올린 뒤 다시 확인한다.

2.8 도구와 타깃을 확인한다

다음은 Ubuntu/WSL Bash의 기본 설치 예시다. ROS 2는 선택한 배포판의 공식 설치 안내를 따른다. 원본의 Foxy/Humble 문구를 섞어 하나의 설치 절차로 만들지 않는다.

```
sudo apt update
sudo apt install build-essential cmake ninja-build git \
    gcc-arm-none-eabi libnewlib-arm-none-eabi \
    libstdc++-arm-none-eabi-newlib python3
mkdir -p ~/robot-dev/deps ~/robot-dev/logs
uname -a
cmake --version
arm-none-eabi-gcc --version
python3 --version
```

PC용 C 연습에는 gcc를, 펌웨어에는 arm-none-eabi-gcc를 쓴다. 이 책의 보드는 Pico 2 W, 타깃은 pico2_w, CPU 경로는 RP2350 Arm이다. RP2040용 예제나 RISC-V 빌드의 성공을 같은 검증으로 기록하지 않는다.

SDK는 공식 저장소에서 받는다. 다음 명령은 해당 폴더가 없을 때 실행한다. 기존 의존성을 지우고 다시 받지 않는다.

```
cd ~/robot-dev/deps
git clone https://github.com/raspberrypi/pico-sdk.git
cd pico-sdk
git submodule update --init --recursive
git rev-parse HEAD
git submodule status --recursive
export PICO_SDK_PATH="$HOME/robot-dev/deps/pico-sdk"
test -f "$PICO_SDK_PATH/src/boards/include/boards/pico2_w.h"
```

처음 받은 커밋은 후보 버전이다. 검증을 마친 뒤 그 커밋을 고정한다. 새 터미널에서는 PICO_SDK_PATH를 다시 설정한다. 값이 비어 있으면 파일이 있어도 CMake가 찾지 못한다.

2.9 성공한 조합을 고정한다

원본 CMake는 FreeRTOS의 RP2350_ARM_NTZ 포트를 참조한다. 의존성 버전에 따라 import 파일 경로가 달라질 수 있으므로 실제 checkout에서 확인한다. 경로 오류를 해결하려고 다른 CPU 포트를 임의로 선택하지 않는다. libmicroros.a도 어떤 CPU·ABI·ROS 배포판·타입·엔티티 설정으로 생성했는지 알아야 한다.

항목	기록할 값	성공 증거
OS·ROS 배포판	OS 버전, ROS_DISTRO	설치·CLI 확인
SDK·FreeRTOS	URL, 커밋, submodule	최소 타깃 빌드
컴파일러	실행 경로, 버전, ABI 옵션	CMake 출력·ELF
micro-ROS 라이브러리	생성 커밋·설정·해시	Agent 왕복
Agent	배포판, 커밋 또는 이미지 digest	직렬·UDP 로그

Ubuntu 24.04와 Jazzy는 호스트 설치 후보지만 Pico 2 W·RTOS 통합을 자동 보장하지 않는다. 라이브러리와 Agent는 같은 ROS 배포판 계열에서 시작하고, 실패할 때 한 조건씩 바꾼다. 최신이라는 이유로 모든 의존성을 동시에 갱신하면 성공과 실패의 원인을 잃는다.

2.10 실습: 환경 확인표를 만든다

1. OS·터미널·보드 모델·데이터 통신 가능한 USB 케이블을 기록한다.
2. 버전 명령의 결과와 실행 경로를 logs/environment.txt에 저장한다.
3. SDK 커밋과 submodule 상태를 기록한다.
4. BOOTSEL을 누른 채 연결하고 RP2350 볼륨이 나타나는지 확인한다. 모터는 연결하지 않는다.
5. SDK → RTOS → 직렬 → Wi-Fi의 검증 칸을 만들고 미실시는 그대로 둔다.

정상 결과는 도구 경로가 확인되고 미확정 항목의 이유가 적힌 환경표다. 장치가 보이지 않으면 케이블·포트·BOOTSEL 순서를 확인한다. 장치 권한 문제 때문에 모든 빌드 명령을 sudo로 실행하면 파일 소유권까지 달라질 수 있다. USB 접근 문제와 컴파일 문제를 따로 해결한다.

2.11 작업 폴더와 터미널을 정리한다

프로젝트 폴더 안에 SDK를 무작정 복사하면 어느 파일이 저자의 코드이고 어느 파일이 외부 의존성인지 구분하기 어렵다. 다음은 소스·의존성·산출물·측정 기록을 분리한 예다. 폴더 이름 자체가 필수 규약은 아니지만 각 폴더의 역할은 유지한다.

~/robot-dev/sample/	샘플에서 저장한 C 예제
~/robot-dev/deps/pico-sdk/	외부 SDK
~/robot-dev/logs/	환경·빌드·실행 기록
/tmp/robot-book-ch04/	다시 생성할 수 있는 빌드 결과

빌드 폴더는 소스에서 다시 만들 수 있다. 하지만 실물 관찰 기록은 다시 빌드한다고 돌아오지 않는다. 사진·측정값·사용한 부품 판본·보정값은 로그 폴더와 프로젝트 기록에 보존한다. “정리”라는 이름으로 두 종류를 함께 삭제하지 않는다.

WSL에서는 Linux 파일 경로가 /home/사용자/... 이고 Windows에서 같은 파일에 접근하는 경로는 WSL 공유 경로다. Bash에 Windows의 역슬래시 경로를 그대로 넣거나 PowerShell에 Bash의 환경 변수 문법을 넣으면 다른 의미가 된다. 책의 코드 블록에 bash라고 표시한 명령은 Ubuntu 또는 WSL Bash에서 실행한다. 복사하기 전에 현재 프롬프트와 pwd의 결과를 확인하는 습관이 의외로 많은 시간을 절약한다.

```
pwd
printf 'SDK=%s\n' "$PICO_SDK_PATH"
command -v cc
command -v cmake
command -v arm-none-eabi-gcc
```

`command -v`는 실행할 프로그램의 위치를 보여 준다. 버전만 같아도 여러 경로에 설치된 도구가 번갈아 선택될 수 있으므로 위치까지 기록한다. 출력이 없으면 그 터미널의 PATH에서 찾지 못한 것이다. 파일이 없다는 결론을 내리기 전에 다른 셸에만 설정한 경로인지 확인한다.

환경 변수를 영구 설정하기 전에 확인할 것

`export PICO_SDK_PATH=...`는 현재 셸과 그 셸이 실행한 프로그램에 적용된다. 이미 열려 있는 다른 터미널의 값은 바뀌지 않는다. 처음에는 현재 셸에서만 설정하여 빌드를 확인하고, 성공한 경로를 반복 사용할 때 셸 초기화 파일에 추가한다. 기존 설정에 같은 변수를 여러 번 덧붙이면 앞의 값이 뒤에서 덮어써지는 문제를 만들 수 있다.

환경 변수에 비밀번호를 넣고 모든 변수를 통째로 출력하는 방식으로 문제를 진단하지 않는다. 이장에서 필요한 것은 SDK 경로와 컴파일러 경로다. 네트워크 정보가 필요한 15장에서도 설정의 존재와 적용 여부를 확인하고 실제 비밀 값은 원고와 로그에서 제외한다.

2.12 의존성을 고정한다는 말의 정확한 의미

Git 저장소의 브랜치 이름은 움직이는 이름이다. 오늘의 main과 다음 달의 main이 같은 파일을 가리킨다는 보장은 없다. 커밋 해시는 특정 소스 상태를 식별한다. 다만 하위 모듈이 있는 프로젝트에서는 상위 저장소 해시만 기록하고 하위 모듈을 빠뜨리면 일부 코드가 비어 있거나 다른 버전일 수 있다.

`git submodule status --recursive`의 출력은 하위 모듈의 커밋과 상태를 확인하는 자료다. 의도한 커밋과 다른 표시가 있으면 바로 덮어쓰지 말고 현재 변경 여부를 먼저 확인한다. 이미 실습 중인 폴더에서 의존성을 갱신해야 한다면 성공한 조합을 별도 기록한 뒤 새 빌드 폴더에서 비교한다.

정적 라이브러리 `libmicroros.a`는 소스만큼 중요한 의존성이다. 파일명이 같아도 다른 CPU 옵션과 메시지 설정으로 생성했을 수 있다. 따라서 “파일이 있다”와 “이번 펌웨어에 맞는다”는 별개의 판단이다. 적어도 생성 출처, ROS 배포판, 대상 아키텍처, 빌드 설정, 파일 해시를 기록해야 한다. 메시지 타입이나 정적 엔티티 한도를 바꿨다면 같은 라이브러리를 그대로 사용할 수 있는지도 다시 확인한다.

변경	새로 확인할 부분	앞선 결과의 재사용 범위
설명 문장만 수정	문서 빌드와 링크	펌웨어 결과 재사용 가능
C 컴파일러 변경	구성·컴파일·링크·최소 실행	실물 결과 그대로 승계하지 않음
SDK 커밋 변경	드라이버와 무선 초기화	PC 논리 실습은 독립적으로 유지
메시지 타입 변경	라이브러리 타입 지원과 수신 용량	이전 토픽 왕복은 새 타입 증거가 아님
서보 모델 변경	전원·보장·기구 범위	소프트웨어 문법 검사만 유지

이 표가 의미하는 것은 매번 모든 것을 처음부터 하라는 뜻이 아니다. 변경된 조건과 관련된 증거를 다시 확보하라는 뜻이다. 변경 범위를 설명할 수 있으면 필요한 검증 범위도 줄일 수 있다.

2.13 첫 실습: 환경 기록과 PC 예제를 함께 만든다

다음 명령은 버전 정보를 선택해서 저장한다. 오류 출력까지 기록하는 이유는 “도구를 찾지 못했다”는 결과도 환경 준비 상태를 설명하기 때문이다. 실습에서는 출력 파일을 만든 뒤 내용을 직접 읽는다.

```
mkdir -p ~/robot-dev/logs
{
  date -Iseconds
  uname -a
  cat /etc/os-release
  command -v cc
  cc --version
  command -v cmake
  cmake --version
  command -v arm-none-eabi-gcc
  arm-none-eabi-gcc --version
} > ~/robot-dev/logs/environment.txt 2>&1
cat ~/robot-dev/logs/environment.txt
```

그다음 3장 「직접 실행하는 논리 실습」의 전체 C 코드를 ~/robot-dev/sample/ch03_command.c로 저장한다. 폴더가 없다면 먼저 mkdir -p ~/robot-dev/sample로 만든다. 공개 샘플에는 전체 책의 예제 묶음이 포함되지 않으므로 다음 독립 컴파일 명령을 사용한다.

```
cd ~/robot-dev/sample
cc -std=c11 -Wall -Wextra -Werror ch03_command.c -o ch03_command
./ch03_command
```

정상이면 입력별 판정 뒤 command checks passed가 표시된다. 이 코드는 입력과 결과를 assert로 검사한다. PC 컴파일러와 명령 검증 논리를 확인한 것이며 Pico 실물 검증을 뜻하지 않는다. NDEBUG 옵션으로 검사를 끈 결과는 같은 검증으로 취급하지 않는다.

이 단계가 실패하면 아직 USB 케이블을 바꿀 이유가 없다. 파일 경로와 C 컴파일러, 복사한 코드의 누락을 확인한다. 반대로 PC 예제는 통과하지만 BOOTSEL 볼륨이 보이지 않으면 컴파일러 재설치보다 케이블과 연결 상태를 먼저 본다. 확인한 계층 바로 다음 경계를 조사하는 것이 문제 해결의 기본 순서다.

2.14 환경 준비를 완료했다고 판단하는 기준

환경표의 항목은 설치 여부와 검증 여부를 분리한다. 예를 들어 Arm 컴파일러가 설치되었어도 4장의 UF2 빌드 전에는 펌웨어 빌드 칸을 미검증으로 둔다. 통합 firmware를 올리지 않았으면 FreeRTOS와 micro-ROS 칸도 미검증이다. 빈칸을 성공으로 해석하지 않도록 상태 이름을 통일한다.

권장 상태는 준비 전, 준비 완료, 실행 통과, 실패, 해당 없음이다. 실패에는 최초 오류와 재현 명령을 남긴다. 해당 없음은 PC 예제의 USB 확인처럼 그 검사에 장치가 필요 없는 경우다. “아직 하지 않음”을 해당 없음으로 바꾸면 진행 상황을 잘못 읽게 된다.

장 끝의 산출물은 도구 버전 화면 캡처 하나가 아니라 환경표, PC 실습 검사 결과, 다음 확인 단계다. 이 세 가지가 준비되어 있으면 다른 컴퓨터로 옮겼을 때도 무엇부터 다시 확인할지 알 수 있다.

생각을 넓히는 실험

SDK 경로를 고의로 비우는 실험은 실제 파일을 삭제하지 않고도 구성 실패를 관찰할 수 있다. 현재 셸에서만 변수를 해제하고 새 빌드 폴더로 구성해 보자. 최초 오류를 적은 다음 원래 경로를 다시 설정한다. 기존 빌드 폴더에는 캐시가 남을 수 있으므로 이 실험에서는 폴더를 구분해야 한다.

AI 도구에 오류를 질문할 때는 운영체제, 보드, 타깃, 도구 경로, 실행 명령, 최초 오류를 제공한다. 비밀번호와 전체 환경 덤프는 제공하지 않는다. 답변이 RP2040용 포트나 다른 ROS 배포판으로 조건을 바꾸고 있지 않은지 확인한다. 설명이 그럴듯하더라도 이 책의 환경표와 다른 전제를 쓰면 곧바로 적용하지 않는다.

AI와 함께 문제 해결하기

개발 환경 문제는 긴 설치 명령을 다시 실행하기 전에 어디에서 실패했는지 구분하는 데서 시작한다. AI에게 실제 환경표와 최초 오류를 주고, 작은 확인을 하나씩 수행하며 원인을 좁혀 보자.

문제 1. SDK 구성 오류의 원인 후보를 좁히기

PC 예제는 통과했지만 Pico 펌웨어 구성에서 SDK import 오류가 발생했다. 오류를 재현한 작업 경로와 명령을 적고, AI가 제안한 확인 순서를 실제 결과로 수정해 보자.

함께 제공할 자료: 환경표, 현재 작업 경로, 구성 명령, 최초 오류 주변 로그. 오류가 없다면 본문의 가상 상황으로 진행한다.

예시 프롬프트

Pico 2 W 개발 환경을 점검 중이다.
 환경·작업 경로·실행 명령·최초 오류: [자료]
 PC 예제 검사 결과: [실제 결과 또는 미실행]
 실패 단계를 먼저 구분하고 원인 후보 세 개를 제시해 줘.
 후보를 구분할 읽기 전용 확인을 하나씩 제안해 줘.
 확인 결과를 받기 전에 재설치나 파일 삭제를 해결책으로 확정하지 마.

직접 확인하고 남길 것: 후보별 근거와 확인 결과를 표로 남긴다. 하나의 관찰로 어떤 후보를 제외했는지 설명한다.

문제 2. AI와 후보 보드의 선택 근거 검토하기

1장의 임무 카드에 맞춰 Pico 2 W와 다른 계열 보드 두 개를 비교하자. 필수 조건에 맞지 않는 후보를 먼저 제외하고, 남은 후보의 총비용과 이식 작업을 비교한다. 처음의 선택을 바꾸어도 좋지만 무엇을 확인하고 판단을 바꿨는지 남긴다.

함께 제공할 자료: 임무 카드, 장치 목록, 보드 예산, 공식 사양·가격 URL과 확인일, 보유 장비, 개발 환경 기록. 아직 모르는 값은 빈칸으로 둔다.

예시 프롬프트

내 로봇의 임무·장치·예산·보유 장비: [자료]
 후보는 Pico 2 W와 [보드 A], [보드 B]다.
 공식 사양과 가격 출처·확인일: [자료]
 먼저 빠진 필수 조건을 질문하고, 핀·전원·메모리·통신을 비교해 줘.
 단품 가격과 추가 부품을 포함한 총비용을 구분해 줘.
 micro-ROS 지원은 보드·SDK·배포판·전송 조합별 근거를 제시해 줘.
 확인하지 못한 가격과 지원 상태는 추정하지 말고 미확인으로 남겨 줘.
 내가 선택하면 첫 빌드와 메시지 왕복의 검증 계획을 함께 검토해 줘.

직접 확인하고 남길 것: AI가 제시한 URL을 직접 열어 모델·메모리·단품 조건을 대조한다. 최종 후보와 제외 이유, 총비용, 핀 배정표, 추가 이식 작업을 남긴다. 선택한 보드의 SDK·툴체인·라이브러리는 정확한 식별자와 실제 검사 결과를 연결하며, 아직 실행하지 않은 조합을 검증 완료로 표시하지 않는다.

문제 3. Windows와 WSL에서 보드 상태 추적하기

BOOTSEL 저장장치와 실행 중 USB 장치를 관찰할 위치를 나누어 보자. 장치가 보이지 않는다는 한 문장을 어떤 상태와 경로의 문제인지 설명할 수 있게 만든다.

함께 제공할 자료: 사용 OS·터미널, 보드 모드, Windows와 WSL 각각에서 관찰한 장치 정보.

예시 프롬프트

Windows와 WSL에서 Pico 2 W 접근 경로를 비교한다.
 보드 모드와 각 환경에서 관찰한 정보: [자료]
 저장장치 표시와 실행 중 통신 포트를 구분해 질문해 줘.
 내 자료로 확인된 연결과 추가 확인이 필요한 경계를 그려 줘.
 관찰되지 않은 장치 이름이나 포트 경로는 가정하지 마.
 보드가 없으면 실행 결과 대신 관찰 계획만 작성해 줘.

직접 확인하고 남길 것: 보드 상태·관찰 장소·관찰 도구·결과를 대응시킨다. PC 예제 통과를 보드 연결 확인으로 대신하지 않는다.

2장 마무리

실습 완료 체크리스트

- ☐ 기준 보드와 대안 보드의 선택 이유를 공식 사양·가격 근거로 설명했다.
- ☐ 단품 가격과 총비용, GPIO와 전원 핀, SRAM과 플래시를 구분했다.
- ☐ 호스트·보드·USB 접근 장소를 구분했다.
- ☐ 버전·경로·미검증 항목을 기록했다.
- ☐ 개인 접속 정보는 원고와 버전 관리에서 제외했다.

핵심 정리

보드는 로봇의 요구에 맞춰 선택한다. Pico 2 W는 이 책의 기준이며 다른 보드에서도 제어 구조를 재사용할 수 있다. 하드웨어 사양·드라이버·통신 경로가 바뀌면 그 경계를 다시 확인한다. 재현 환경은 이렇게 선택한 조건과 버전 목록, 성공 증거의 연결이다. 다음 장에서는 보드 없이 검사할 수 있는 C 제어 로직을 먼저 만든다.

참고 자료

- Pico 2 W 공식 데이터시트: 핀·전원·기구·무선
<https://datasheets.raspberrypi.com/picow/pico-2-w-datasheet.pdf>
- Pico 계열 공식 안내
<https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>
- Pico SDK 공식 안내
https://www.raspberrypi.com/documentation/microcontrollers/c_sdk.html

- ROS 2 Jazzy Ubuntu 설치

<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

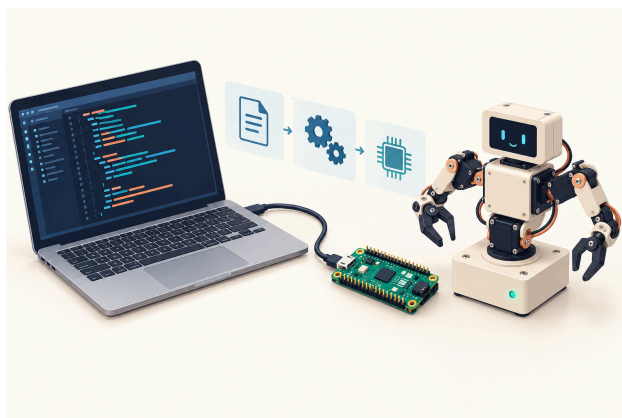
- 기준 CMake

https://github.com/freshmea/micro_ros_pico_dev/blob/393f96b6e484b5c70245b821dddd3aca2d7cc1fc/CMakeLists.txt

03

CHAPTER

로봇 제어에 필요한 Embedded C



학습 목표

- C·Rust의 특징과 실제 임베디드 적용 사례를 구분한다.
- 구조체·버퍼의 수명과 실패 시 상태 보존을 설명한다.

이 장의 학습 흐름

STEP 01
언어 선택

STEP 02
상태와 수명

STEP 03
입력 검사

완료 기준

- 이 프로젝트의 C 선택 이유를 설명하고 PC 예제를 실행한다.
- 범위 밖 입력이 기존 상태를 바꾸지 않는지 확인한다.

이번 장의 결과물 입력 검사를 갖춘 명령 구조

팔을 30도 움직이라는 요구를 코드로 옮기려면 숫자뿐 아니라 관절 번호·방향·허용 범위·실행 상태가 필요하다. 이 장은 C 문법 전체보다 로봇에서 자주 생기는 오류를 막는 자료 표현과 함수 경계를 익힌다. 보드를 연결하기 전 PC에서 입력을 바꾸어 보는 것이 첫 실습이다.

학습 목표와 완료 기준

C와 Rust가 펌웨어 개발에 쓰인다는 점과 이번 프로젝트의 C 선택 이유를 설명한다. MCU용 Rust 예제와 Microduck의 Linux 기반 Rust 프로그램을 구분한다. 정수와 단위, 구조체·enum, 포인터와 버퍼 수명을 설명한다. 입력 검사와 상태 변경을 분리한 C 프로그램을 컴파일하고 정상·경계·거부 입력을 확인하면 완료다. PC 검사 결과는 GPIO·PWM 동작 검증과 구분한다.

작은 숫자 오류가 실제 동작 오류가 되는 이유

일반적인 화면 프로그램에서 181을 잘못 입력하면 이상한 문장이 나타날 수 있다. 관절 프로그램에서는 같은 값이 기구를 끝까지 움직이라는 요청이 될 수 있다. 따라서 임베디드 C를 배울 때는 문법이 맞는지와 요청이 의미상 올바른지를 함께 본다. 컴파일러는 int에 181을 저장하는 것을 정상으로 판단하지만 그 관절이 181도를 허용하는지는 알지 못한다.

로봇 프로그램에는 최소 세 가지 경계가 있다. 문자열이나 메시지를 숫자로 해석하는 **표현 경계**, 값의 범위와 현재 상태를 확인하는 **정책 경계**, 실제 PWM으로 바꾸는 **장치 경계**다. 이 경계를 한 함수 안에 섞으면 숫자 변환에 실패했는데도 장치가 움직이는 문제가 생기기 쉽다. 이번 장의 예제는 앞의 두 경계를 PC에서 검사하고, 장치 경계는 6장으로 미룬다.

예를 들어 사용자가 “-20”이라는 문자열을 보냈다고 하자. 먼저 문자열 전체가 정수 표현인지 검사한다. 다음에는 논리 관절각 -20도가 허용 범위에 있는지 확인한다. 마지막으로 현재 상태가 정지 중이면 요청을 거부한다. 숫자가 올바르다고 항상 실행할 수 있는 것도 아니고, 실행할 수 있는 상태라고 모든 숫자를 받아도 되는 것도 아니다.

값을 잘 표현하면 조건문이 짧아진다

`int state = 2;`라는 코드는 2의 의미를 다른 곳에서 찾아야 한다. `Mode mode = STOPPED;`는 이름만으로 의미를 읽을 수 있다. 하지만 enum도 값의 무결성을 자동으로 보장하지 않는다. 외부 정수를 enum으로 캐스트하기 전에 허용된 종류인지 검사해야 한다. 타입은 의도를 드러내고 검사는 경계를 지킨다.

구조체는 함께 움직여야 할 값을 묶는다. 요청 ID, 목표각, 지속 시간을 각각 전역 변수로 저장하면 세 값이 서로 다른 요청에서 온 조합이 될 수 있다. Command 하나로 묶고 유효성을 확인한 뒤 한 번에 전달하면 “어느 요청의 목표인가”를 잃지 않는다. 12장의 큐도 바로 이 단위를 복사한다.

3.1 펌웨어를 만드는 언어: C와 Rust

보드를 여러 후보 중에서 골랐듯이 펌웨어 언어에도 선택지가 있다. **C로 펌웨어를 작성할 수 있고 Rust로도 작성할 수 있다.** 이번 프로젝트의 실습은 C로 진행하지만, C만 배워야 임베디드 장치를 만들 수 있다고 생각할 필요는 없다. 언어를 선택할 때는 표현 방식과 함께 대상 칩의 지원, 드라이버, 실행 환경, 기존 코드와 팀의 경험을 살펴본다.

C는 컴파일한 코드를 통해 메모리와 주변장치에 접근하는 시스템 프로그래밍 언어다. 이 책에서는 구조체와 함수로 상태·입력·출력을 표현하고, 포인터와 배열의 길이·수명을 직접 관리한다. Pico SDK와 참고 로봇 코드의 C API를 따라가며 GPIO·PWM·통신 동작이 만들어지는 경로를 관찰하기에 적합하다.

Rust도 대상 CPU에서 실행할 코드를 만드는 시스템 프로그래밍 언어다. **소유권과 빌림**이라는 규칙으로 데이터의 사용과 수명을 표현한다. 안전한 Rust 코드에서는 해제된 데이터를 참조하거나 잘못된 공유로 데이터 경쟁을 일으키는 여러 오류를 컴파일 단계에서 막을 수 있다. 필수 가비지 컬렉터에 의존하지 않으므로 자원이 제한된 장치에서도 사용하는 경로가 있다. Rust 임베디드 공식 안내와 소유권 설명에서 이 설계 방향을 확인할 수 있다.

관점	C에서 배우는 방식	Rust에서 만나는 방식
데이터의 수명	포인터가 가리키는 저장 공간이 언제까지 유효한지 작성자가 추적한다.	소유권과 빌림 규칙으로 수명 관계를 표현하고 컴파일러의 검사를 받는다.
장치 접근	SDK·드라이버의 함수와 자료형을 사용한다.	대상 칩의 HAL과 드라이버 crate를 사용하거나 C 라이브러리와 연동한다.
실패 표현	반환값·오류 코드와 출력 인자의 계약을 정한다.	Result·Option 같은 타입으로 성공·실패·값 없음의 처리를 드러낼 수 있다.
빌드 구성	이 책은 CMake와 Arm 컴파일러, Pico SDK를 연결한다.	보통 Cargo로 패키지·의존성·빌드를 관리하고 대상 CPU·링커·실행 기반을 설정한다.
실행 구조	반복 루프, 인터럽트, FreeRTOS 태스크 등을 선택한다.	반복 루프·인터럽트·비동기 실행기 등 대상 환경에 맞는 구조를 선택한다.

HAL은 Hardware Abstraction Layer로, 하드웨어 기능을 일정한 인터페이스로 다루도록 돕는 계층이다. crate는 Rust 코드의 빌드 단위이며 라이브러리나 실행 프로그램이 될 수 있다. 언어 이름을 바꾸었다고 GPIO 초기화나 전원 조건이 사라지는 것은 아니다. 같은 LED를 켜더라도 어떤 핀과 드라이버를 사용할지는 여전히 보드 문서에서 확인한다.

Rust의 메모리 안전성은 로봇 전체의 동작 안전성과 구분한다. 유효한 정수 180이 관절의 기구 범위를 넘는지, 통신이 끊겼는데 마지막 명령을 계속 실행해도 되는지는 언어가 결정하지 않는다. 또한 하드웨어 레지스터·외부 C 함수 등과 만나는 `unsafe` 경계에는 별도의 계약 검토가 필요하다. `unsafe`가 다른 모든 검사를 끈다는 뜻도 아니다. 구체적인 범위는 Rust 공식 `unsafe` 설명을 따른다. 이 장에서 배울 입력 범위와 상태 변경의 책임은 두 언어 모두에 필요하다.

3.2 Rust 펌웨어는 이미 진행 중인 개발 분야다

Rust 기반 펌웨어는 가능성을 이야기하는 단계에만 머물러 있지 않다. MCU용 하드웨어 지원, 드라이버, 비동기 실행, 부트로더 등 여러 영역에서 연구와 개발이 활발히 이어지고 있다. [Embedded Rust Book](#)은 학습 체계를 제공하고, [Embassy](#)는 임베디드 Rust 애플리케이션을 위한 라이브러리와 실행 구조를 제공한다. 이 책이 C로 실습한다는 사실은 Rust 펌웨어의 가능성을 제한하는 선택이 아니다.

운영체제 없이 Rust를 실행하는 경로

PC용 Rust 프로그램에서 쓰는 표준 라이브러리 `std`의 일부 기능은 운영체제 서비스를 전제로 한다. MCU에서는 `#![no_std]` 환경을 사용하여 그러한 전제를 줄일 수 있다. 이 경우에도 기본 언어 기능과 `core`를 사용할 수 있으며, 시작 코드·인터럽트·주변장치·메모리 배치는 대상 칩의 지원 코드와 함께 구성한다. 동적 할당이 필요하면 그에 맞는 별도 설정이 필요하다. 자세한 범위는 [Embedded Rust의 no_std](#) 안내에 정리되어 있다.

따라서 “Rust는 Linux가 있어야 한다”도, “Rust이면 운영체제 없이 아무 보드에서나 실행된다”도 맞지 않는다. 같은 언어라도 MCU에서 직접 실행할 펌웨어와 Linux에서 실행할 프로그램은 빌드 대상과 사용할 수 있는 기능이 다르다.

Pico 2 계열에서 확인할 수 있는 실제 자료

[Embassy의 embassy-rp](#) 문서는 RP2040과 RP235x용 HAL을 제공한다고 설명한다. 공식 저장소의 RP235x 예제 구성에는 RP2350 대상과 필요한 의존성이 들어 있다. 같은 예제 폴더의 무선 보드 LED 예제에서는 `no_std`와 CYW43 드라이버를 함께 확인할 수 있다.

여기서도 2장의 보드 선택 지식이 도움이 된다. 일반 GPIO의 LED 예제와 무선 칩에 연결된 LED 예제는 적용 대상이 다르다. 파일 이름에 `blinky`가 있다는 이유만으로 어떤 Pico에나 같은 코드를 올리지는 않는다. CPU 지원, 보드 핀, 무선 칩, 예제가 요구하는 파일과 업로드 방법을 차례로 읽는다.

[Embassy](#)는 `async`와 `await`를 활용하는 실행 방식을 제공한다. 작업이 기다리는 동안 같은 실행기의 다른 작업이 진행할 수 있도록 구성하는 방식이다. 이를 FreeRTOS의 태스크마다 별도 스택을 두는 실행 방식과 같은 것으로 해석하지 않는다. 오래 걸리는 계산이나 대기를 어떻게 나눌지, 우선순위와 장치 소유권을 어떻게 정할지는 선택한 실행 기반에 맞춰 검토한다.

2026년 9월 30일 조사에서 확인한 **Embassy** 소스 커밋은 9월 29일 UTC의 변경이다. 문서뿐 아니라 현재 진행되는 구현을 확인할 수 있다는 뜻이다. 이 사실을 모든 보드·드라이버·ROS 연동이 같은 수준으로 지원된다는 뜻으로 확대하지 않는다. Rust로 Pico 펌웨어를 만드는 경로와 이 책의 C·FreeRTOS·micro-ROS 통합을 Rust로 옮기는 작업은 따로 검증한다.

3.3 제품 사례: Microduck에서 Rust가 맡은 일

최근 공개된 로봇 제품 중 Rust 활용을 살펴볼 사례가 **Pollen Robotics의 Microduck**이다. 제조사 공식 제품 페이지는 2026년 8월 27일 사전 주문 개시를 안내한다. 2026년 9월 30일 확인 시점의 제품·공개 소프트웨어 사례로 소개하며, 독자에게 이미 배송 완료된 제품으로 표현하지 않는다.

공식 **microduck** 저장소는 로봇 안에서 실행되는 소프트웨어를 Rust workspace로 구성한다. **robotd**는 제어 루프와 모터 버스를 맡고, **configd**는 연결 설정을, **updaterd**는 업데이트를 담당한다. Rust가 학습용 문법 예제뿐 아니라 실제 로봇 제품의 제어와 운영 소프트웨어를 작성하는 데 쓰인다는 점을 보여 준다.

소스에서 언어와 실행 환경을 구분하는 방법

2026년 9월 30일 확인한 커밋 `f0d934e7`을 기준으로 다음 네 파일을 읽으면 주장을 직접 대조할 수 있다.

공식 소스	직접 확인할 항목	여기서 알 수 있는 것
루트 <code>Cargo.toml</code>	workspace에 포함된 Rust 패키지	로봇 서비스와 도구가 하나의 Rust 프로젝트 구성 안에 있다.
<code>robotd/Cargo.toml</code>	제어 데몬의 의존성과 Tokio 설정	보드의 제어 소프트웨어가 Rust 패키지로 구성되어 있다.
<code>robotd/src/main.rs</code>	std, Unix 소켓, 제어 모듈 사용	설명 문서에 더해 실제 Rust 구현과 OS 의존성을 읽을 수 있다.
<code>.cargo/config.toml</code>	aarch64-unknown-linux-gnu 대상과 보드 설명	확인한 프로그램은 RK3566 기반 Linux 환경을 대상으로 빌드한다.

각 파일은 **workspace** 구성, **robotd** 구성, 제어 프로그램 소스, 크로스 빌드 설정에서 확인할 수 있다. 여기서 데몬은 장치 안에서 계속 실행되며 특정 서비스를 맡는 프로그램을 뜻한다.

이 차이는 펌웨어라는 말을 이해하는 데 중요하다. 제품의 내장 소프트웨어나 업데이트 묶음을 넓게 펌웨어라고 부르는 경우가 있지만, 기술적으로는 **MCU에 직접 올리는 펌웨어와 임베디드 Linux에서 실행하는 제어 프로그램**의 실행 조건을 구분해야 한다. Microduck에서 확인한 Rust 코드는 후자의 사례다. 앞 절의 Embassy RP235x 예제는 전자의 경로를 보여 준다. 두 사례를 함께 보면 Rust가 작은 MCU부터 Linux 기반 로봇까지 쓰일 수 있다는 범위를 정확히 이해할 수 있다.

또한 이 자료만으로 모터 내부 MCU의 펌웨어까지 모두 Rust라고 단정할 수는 없다. Rust 제어 프로그램이 모터에 명령을 보내는 것과 모터 안의 프로그램을 Rust로 작성했다는 것은 다른 주장이다. 외부 라이브러리와 운영체제까지 전부 같은 언어라고 해석하지도 않는다. 실제 제품의 기술 선택을 읽을 때는 “어느 장치의 어느 계층이 그 언어로 작성되었는가”를 확인하는 습관이 필요하다.

이 책으로 가져올 설계 관점

Microduck의 아키텍처 문서는 제어와 연결 설정, 업데이트의 책임을 나누는 설계를 설명한다. 이 책에서는 훨씬 작은 로봇을 만들지만 같은 질문을 던질 수 있다. 누가 모터를 소유하는가, 통신이나 화면 처리가 실패해도 어떤 제어가 남아야 하는가, 잘못된 변경을 어떻게 되돌릴 것인가 하는 질문이다.

Rust 제품 사례를 보는 목적은 Microduck의 코드나 학습된 동작을 Pico 2 W에 그대로 올리는 데 있지 않다. 하드웨어·메모리·운영체제가 다른 프로젝트에서도 책임 분리와 실패 대응이라는 공통 설계 원칙을 읽는 데 있다. 이 사례의 소스 확인은 본문 집필을 위한 조사이며 Microduck의 실물 동작을 이 책에서 시험한 결과는 아니다.

3.4 이번 프로젝트에서 C를 사용하는 이유

이 책은 C와 Rust 중 영구적인 승자를 정하지 않는다. **이번 프로젝트의 구현 언어는 C이며, Rust는 후속 학습과 다른 프로젝트에서 선택할 수 있는 펌웨어 언어다.** 지금 C를 선택한 이유는 다음과 같이 책의 목표와 연결된다.

첫째, 기존 로봇 저장소와 Pico SDK의 C API를 같은 표현으로 따라갈 수 있다. 4장의 최소 펌웨어에서 시작하여 주변장치 드라이버와 RTOS·통신 경계로 나아갈 때 언어 간 변환을 추가하지 않아도 된다. 빌드 환경의 이름이 C/C++ SDK라고 해서 이번 실습의 애플리케이션을 C++로 작성하는 것도 아니다. 해당 장의 C 코드와 빌드 설정을 기준으로 읽는다.

둘째, 이 책은 자료의 길이·수명, 실패 시 상태 보존, 단위와 범위를 독자가 직접 설명하도록 구성했다. C에서 포인터가 유효한 기간을 추적해 본 경험은 Rust의 소유권과 빌림을 배울 때도 도움이 된다. 반대로 Rust의 소유권 관점을 먼저 접하면 C 코드를 읽을 때 “이 버퍼의 주인은 누구인가”라는 질문을 더 명확하게 할 수 있다.

셋째, MCU·RTOS·통신을 처음 연결하면서 두 언어의 빌드 도구와 실행 기반까지 동시에 바꾸면 실패 원인을 나누기 어려워진다. 기본 제작은 하나의 경로로 완료하고, 언어를 바꾸는 실험은 입력 검사 같은 작은 논리부터 시작하는 편이 학습 결과를 비교하기 쉽다.

Rust로 확장할 때 확인할 순서

기본 C 실습을 마친 뒤 Rust를 배우고 싶다면 먼저 자료형·소유권·빌림·Result를 익힌다. 그다음 이 장의 입력 검사 계약을 PC용 Rust 함수로 표현해 정상·경계·실패 사례를 비교한다. 반환값뿐 아니라 거부된 요청이 기존 상태를 바꾸지 않는지도 함께 본다. 같은 요구를 두 언어로 표현하는 학습

이며 C 파일을 자동 번역한 뒤 같은 의미라고 가정하는 과정은 아니다.

이후 보드의 공식 또는 프로젝트 제공 최소 Rust 예제를 읽고 LED·입력·타이머·버스를 하나씩 확인한다. 통신을 추가할 때는 Rust 드라이버와 기존 C 라이브러리를 잇는 FFI가 필요한지, 별도의 통신 구현을 사용할지 먼저 조사한다. FFI는 서로 다른 언어로 작성한 함수를 연결하는 인터페이스다. 데이터 배치, 메모리 소유권, 콜백 수명, 동기화 계약을 새로 확인해야 한다.

이 책은 Rust용 micro-ROS 통합 타깃이나 완성 펌웨어를 제공한다고 주장하지 않는다. 대신 독자가 다른 언어의 가능성을 알고 공식 예제와 지원 범위를 검토할 수 있는 출발점을 제공한다. 이제 C를 사용하여 같은 설계 질문을 작은 입력 검사 프로그램부터 구체화해 보자.

3.5 숫자에 단위와 범위를 붙인다

30이 각도인지 밀리초인지 모르면 호출을 읽기 어렵다. `angle_deg`, `duration_ms`, `pulse_us`처럼 단위를 이름에 붙인다. 외부 입력은 부호 있는 타입에서 검사한 뒤 변환한다. 음수를 `uint16_t`로 먼저 바꾸면 큰 양수가 되어 원래 의미를 놓친다.

고정 폭 정수는 저장 범위를 분명하게 하지만 곱셈의 중간값도 넘칠 수 있다. 보간에는 넓은 중간 타입이나 범위를 확인한 실수 연산을 사용한다. 비트 마스크는 여러 플래그를 동시에 표현하고 `enum`은 서로 배타적인 상태를 표현하는 데 적합하다.

```
#include <stdint.h>
#include <stdbool.h>

static bool elapsed(uint32_t now_ms, uint32_t start_ms,
                   uint32_t interval_ms) {
    return (uint32_t)(now_ms - start_ms) >= interval_ms;
}
```

unsigned 뺄셈의 순환을 이용한 조각이다. 타이머가 한 바퀴 이상 도는 동안 갱신을 놓친 경우까지 해결하지 않는다. 최대 동작 시간과 확인 간격을 함께 제한해야 한다.

3.6 입력·상태·출력을 나눈다

입력은 외부 사실이고 상태는 처리 중인 내부 결과다. 출력은 장치에 적용할 값이다. 하나의 전역 변수에 섞으면 누가 변경했는지 찾기 어렵다. 다음은 PC에서 실행하는 완전한 예제다. 전체 코드를 `~/robot-dev/sample/control.c`로 저장한다. 폴더가 없다면 `mkdir -p ~/robot-dev/sample`로 만든다. 컴파일 명령은 이 장 뒤의 「실습: 경계를 하나씩 바꾼다」에 있다.

```
#include <assert.h>
#include <stdbool.h>
#include <stdio.h>
```

```

typedef enum { IDLE, MOVING, STOPPED } Mode;
typedef struct { Mode mode; int target_deg; } Robot;

static bool request(Robot *r, int target_deg) {
    if (r == NULL || r->mode == STOPPED) return false;
    if (target_deg < 30 || target_deg > 150) return false;
    r->target_deg = target_deg;
    r->mode = MOVING;
    return true;
}

int main(void) {
    Robot r = { .mode = IDLE, .target_deg = 90 };
    assert(request(&r, 30));
    assert(request(&r, 150));
    assert(!request(&r, -1));
    assert(!request(&r, 151));
    assert(r.target_deg == 150);
    r.mode = STOPPED;
    assert(!request(&r, 90));
    puts("boundary and stop checks passed");
    return 0;
}

```

30~150도는 경계 검사 연습용 가상 범위다. 실제 허용 범위는 6장에서 측정한다. 거부된 입력은 목표값을 바꾸지 않는다. 정지 중 요청도 거부하므로 각도 검사와 상태 정책이 함께 드러난다.

3.7 포인터는 위치와 수명을 함께 생각한다

Robot 포인터는 구조체를 복사하지 않고 갱신하도록 참조한다. 호출자의 변수는 사용이 끝날 때까지 살아 있어야 한다. 지역 배열의 주소를 반환하거나 콜백 수신 버퍼의 포인터만 큐에 넣으면 나중에 읽을 때 이미 무효이거나 다른 메시지일 수 있다.

함수에 배열을 전달할 때는 길이도 전달한다. sizeof(pointer)는 배열 용량을 알려 주지 않는다. const는 읽기 전용이라는 의도를 표현하지만 길이와 수명을 보장하지 않는다. 한국어 UTF-8은 글자 수와 바이트 수가 다르므로 버퍼 용량은 바이트로 계산한다.

3.8 헤더·반환값·짧은 갱신 함수

헤더에는 외부에서 사용하는 타입과 선언을, 소스에는 구현을 둔다. 내부 함수는 static으로 범위를 제한한다. 초기화와 입력 검사처럼 실패할 수 있는 함수는 결과를 반환하고 호출자는 지속·거부·정지를 선택한다. 오류를 출력하고 계속 진행하면 초기화되지 않은 장치를 사용할 수 있다.

긴 동작은 시작 함수와 현재 시각으로 한 단계 진행하는 갱신 함수로 나눈다. 완료까지 while에서 기다리면 같은 흐름의 정지 입력을 읽지 못한다. 10장에서 이 분리를 상태기계로 확장한다.

3.9 정수 변환을 예측하고 검사한다

C에서 타입 변환은 사람이 생각하는 숫자의 의미와 다르게 작동할 수 있다. -1을 부호 없는 16비트 정수에 저장하면 그 타입의 모듈러 규칙에 따라 큰 값이 된다. 그다음 값이 음수인지 검사해도 이미 부호 정보는 사라졌다. 따라서 메시지가 부호 있는 정수라면 원래 타입에서 범위를 검사하고, 실제 하드웨어 레지스터에 넘기기 직전에 필요한 타입으로 변환한다.

곱셈은 최종 저장 변수만 넓다고 안전해지지 않는다. 작은 타입 두 개의 곱을 먼저 계산한 뒤 넓은 타입에 대입하면 중간 연산에서 이미 넘쳤을 수 있다. 밀리초를 마이크로초로 바꾸거나 각도와 보간 비율을 곱할 때 특히 주의한다. 계산 전 피연산자를 적절한 넓은 타입으로 바꾸고 최대 입력에서 결과가 표현 가능한지 계산한다.

```
uint32_t duration_ms = 3000;
uint64_t duration_us = (uint64_t)duration_ms * 1000u;
```

이 예는 단위 변환의 중간 폭을 분명히 한다. 모든 변수를 uint64_t로 만들라는 뜻은 아니다. 프로토콜의 폭, 최대 기간, 연산 비용을 고려해 필요한 위치에서 사용한다. 외부 값이 지나치게 큰 경우에는 타입을 넓히는 것보다 정책상 거부하는 것이 먼저다.

시간 비교도 숫자의 해석 문제다. 32비트 밀리초 시계는 계속 증가하다가 0으로 돌아온다. 단순히 now가 start보다 큰지 비교하면 이 경계에서 틀릴 수 있다. unsigned 뺄셈으로 경과 시간을 구하는 방법은 한 바퀴를 넘는 미관측 구간이 없다는 조건에서 유용하다. 시작·종료 시각을 부호 있는 값으로 바꿔 비교하는 기법과 혼용하지 않는다. 각 기법의 유효 시간 범위를 함께 설명해야 한다.

3.10 함수의 계약과 변경의 원자성

함수 계약은 입력 조건, 성공 시 결과, 실패 시 결과를 정하는 약속이다. 여기서 원자적 변경이라는 말은 여러 코어 사이의 하드웨어 원자 명령을 뜻하지 않는다. 한 함수 호출에서 유효하지 않은 요청 때문에 구조체가 반만 갱신되지 않도록 한다는 의미다. 동시 접근의 원자성은 12장에서 별도로 다룬다.

목표각은 정상인데 지속 시간이 잘못된 요청을 생각해 보자. 목표각부터 저장하고 지속 시간을 검사하면 함수는 실패를 반환하면서 목표각은 바뀐다. 호출자는 실패했으므로 아무것도 변하지 않았다고 생각할 수 있다. 이를 막으려면 지역 변수 candidate에 후보를 만들고, 모든 검사가 끝난 뒤 실제 목적지에 복사한다.

처리 순서	목표각 정상·시간 오류일 때	호출자의 해석
필드마다 검사하며 즉시 저장	일부 값이 바뀜	실패 후 상태가 불명확
모든 필드를 검사한 후 복사	기존 요청 유지	실패는 변경 없음
오류값을 임의 보정하여 저장	다른 요청이 실행됨	보정 사실을 별도 전달해야 함

이 책의 명령 생성 예제는 두 번째 정책을 사용한다. 화면 밝기처럼 명시적으로 제한할 수 있는 값과 관절 명령처럼 거부가 필요한 값을 같은 정책으로 처리할 필요는 없다. 다만 어떤 정책이든 호출자가 결과를 알아야 한다.

반환값을 버리면 경계도 사라진다

bool을 반환하는 함수는 성공 여부만 표현한다. 학습 초기에는 간결하지만 통합 단계에서는 잘못된 각도, 잘못된 시간, 정지 상태, 큐 포화를 구분하기 어렵다. 14장에서는 이 결과를 명령 응답으로 확장한다. 처음부터 복잡한 에러 체계를 만들 필요는 없지만 실패를 성공처럼 진행하지 않는 습관은 처음부터 지킨다.

함수 이름도 계약의 일부다. `parse_angle`은 해석을, `make_command`는 유효한 요청 구성을, `apply_pwm`은 장치 적용을 뜻하도록 나누면 테스트 범위가 분명해진다. `parse` 함수에서 서보를 움직이지 않는다. 테스트 입력 “12x”를 넣었을 때 장치가 잠깐 12도로 움직이는 구조라면 경계가 너무 낮다.

3.11 배열·포인터·문자열을 구분한다

`char text[16]`은 16바이트 저장 공간을 가진 배열이다. `char *text`는 문자 저장 공간을 가리키는 포인터다. 포인터 변수만 선언한다고 문자열 공간이 생기지 않는다. 배열 이름을 함수에 전달하면 대개 첫 원소를 가리키는 포인터로 변환되므로 함수 내부의 `sizeof`만으로 원래 배열 크기를 알 수 없다.

문자열의 마지막에는 NUL 문자 한 바이트가 필요하다. 16바이트 배열에 텍스트 16바이트를 넣고 종결 문자를 추가하면 경계를 넘는다. 반대로 ROS 문자열의 `size`는 일반적으로 유효한 텍스트 길이를 나타내므로 `capacity`와 같은 의미로 사용하면 안 된다. 이 구분은 16장의 실제 메시지 메모리 실습으로 이어진다.

수명도 확인해야 한다. 함수가 종료되면 지역 배열은 더 이상 유효한 저장 공간이 아니다. 주소값이 당장 0으로 바뀌지 않더라도 다시 사용해도 된다는 뜻이 아니다. 같은 이유로 통신 콜백에서 받은 메시지 포인터를 장시간 보관하는 것도 주의해야 한다. 다음 수신이 그 저장 공간을 덮어쓸 수 있기 때문이다.

`const`는 이 포인터를 통해 데이터를 변경하지 않겠다는 의도를 표시한다. 원본이 다른 태스크에서 바뀌지 않는다는 보장도, 포인터가 영원히 유효하다는 보장도 아니다. 읽기 전용 접근, 저장 공간의 수명, 동시 접근은 서로 다른 문제다.

3.12 소스 파일을 나누는 기준

헤더에는 다른 파일이 알아야 할 타입과 함수 선언을 둔다. 구현 파일 내부에서만 쓰는 보조 함수는 `static`으로 제한한다. 관절 범위를 검사하는 함수와 입력 문자열을 해석하는 함수를 서로 다른 파일로 나눌지는 프로젝트 크기에 따라 결정한다. 중요한 기준은 파일 수가 아니라 변경 이유다.

예를 들어 입력이 터미널 문자열에서 ROS 메시지로 바뀌어도 관절 범위 검사는 유지되어야 한다. 터미널 처리에 범위 숫자를 숨겨 놓으면 ROS 경로에서는 같은 검사가 누락될 수 있다. 공통 정책 함수는 입력 수단과 독립적으로 두고, 입력 경로마다 같은 함수를 호출하도록 한다.

전역 변수를 줄인다고 모든 값을 무조건 지역 변수로 만들지는 않는다. 정지 래치나 제어 상태는 여러 호출 사이에 유지되어야 한다. 이런 상태를 구조체에 묶어 소유한 모듈을 명확히 하고 포인터로 전달한다. 상태가 없는 순수 계산 함수와 상태를 바꾸는 함수를 구분하면 단위 검사도 쉬워진다.

3.13 경고와 검사의 역할

컴파일 경고는 실행 전에 발견할 수 있는 문제를 알려 준다. 사용하지 않은 변수, 잘못된 형식 지정자, 암묵적 변환은 단순한 미관 문제가 아닐 수 있다. 예제는 경고를 오류로 다뤄 새 경고를 방지하지 않도록 한다. 경고를 없애기 위해 무조건 캐스트를 추가하면 의도를 드러내기보다 오류를 숨길 수 있다.

`assert`는 프로그래머가 기대한 조건이 맞는지 확인한다. 실패하면 프로그램을 중단하므로 시험에서 문제 위치를 빨리 찾을 수 있다. 반면 외부에서 들어오는 잘못된 요청은 실제 제품에서 정상적으로 발생할 수 있다. 외부 입력 거부를 `assert` 하나에만 맡기지 않고, 일반 조건문과 반환값으로 처리한다. 예제의 `assert`는 그런 함수가 올바르게 거부했는지 검사하는 역할이다.

시험 입력은 정상값 하나에 그치지 않는다. 최소값 바로 아래, 최소값, 중간값, 최대값, 최대값 바로 위를 준비한다. 문자열에는 빈 문자열과 숫자 뒤 문자가 붙은 경우도 넣는다. 실패 후 기존 값이 유지되는지까지 확인해야 변경 계약을 검증할 수 있다.

직접 실행하는 논리 실습

이번에는 단순 정수 인수를 넘어 문자열을 명령 구조체로 바꾼다. 논리각 -45~45도와 지속 시간 20~5000 ms는 PC 학습용 계약이다. 실제 관절의 범위는 보정표로 대체한다.

아래 전체 코드를 `~/robot-dev/sample/ch03_command.c`로 저장한다. Ubuntu 또는 WSL Bash에서 다음과 같이 실행한다. 보드를 연결할 필요는 없다.

```
cd ~/robot-dev/sample
cc -std=c11 -Wall -Wextra -Werror ch03_command.c -o ch03_command
./ch03_command
```

```

#include <assert.h>
#include <errno.h>
#include <inttypes.h>
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    uint32_t id;
    int32_t target_deg;
    uint32_t duration_ms;
} Command;

static bool parse_angle(const char *text, int32_t *out) {
    if (text == NULL || out == NULL || *text == '\\0') return false;
    char *end = NULL;
    errno = 0;
    long value = strtol(text, &end, 10);
    if (errno != 0 || end == text || *end != '\\0') return false;
    if (value < -45 || value > 45) return false;
    *out = (int32_t)value;
    return true;
}

static bool make_command(Command *out, uint32_t id,
                        const char *angle, uint32_t duration_ms) {
    int32_t target;
    if (out == NULL || !parse_angle(angle, &target)) return false;
    if (duration_ms < 20 || duration_ms > 5000) return false;
    Command candidate = {id, target, duration_ms};
    *out = candidate;
    return true;
}

int main(void) {
    const char *inputs[] = {"-46", "-45", "0", "45", "46", "12x", ""};
    const bool expected[] = {false, true, true, true, false, false,
                             false};
    Command active = {0, 0, 1000};
    for (size_t i = 0; i < sizeof(inputs) / sizeof(inputs[0]); ++i) {
        Command before = active;
        bool ok = make_command(&active, (uint32_t)i + 1, inputs[i],
                               1000);
        assert(ok == expected[i]);
    }
}

```

```

    if (!ok) {
        assert(active.id == before.id);
        assert(active.target_deg == before.target_deg);
    }
    printf("input='%s' accepted=%d target=%" PRId32 "\n",
           inputs[i], ok, active.target_deg);
}
assert(!make_command(&active, 99, "0", 0));
assert(!make_command(NULL, 99, "0", 1000));
puts("command checks passed");
return 0;
}

```

PC에서 확인한 출력

```

input='-46' accepted=0 target=0
input='-45' accepted=1 target=-45
input='0' accepted=1 target=0
input='45' accepted=1 target=45
input='46' accepted=0 target=45
input='12x' accepted=0 target=45
input='' accepted=0 target=45
command checks passed

```

코드에서 따라갈 네 지점

첫째, `strtol`을 호출하기 전에 `errno`를 0으로 만든다. 이전 함수가 남긴 오류 상태를 이번 변환의 결과로 오해하지 않기 위해서다. 호출 뒤에는 변환 오류뿐 아니라 숫자를 하나라도 읽었는지, 문자열 끝까지 읽었는지를 확인한다. “12x”를 12로 받아들이지 않는 이유다. 이 예제는 `strtol`이 허용하는 선행 공백을 받아들이지만 후행 문자가 남으면 거부한다. 더 엄격한 문법이 필요하면 공백 정책도 따로 검사한다.

둘째, `long` 값이 논리 범위 안에 있는지 확인한 다음 `int32_t`에 저장한다. 좁은 타입으로 바꾸기 전에 검사하는 순서가 중요하다. 셋째, 목표값과 지속 시간을 모두 확인한 뒤 `candidate`를 복사한다. 넷째, 거부 입력 다음에 이전 목표값이 유지되는지 `assert`로 확인한다. 성공 여부만 확인했을 때 놓칠 수 있는 부분이다.

직접 바꾸어 볼 과제

“0”을 “+0”으로 바꾸면 어떤 결과가 나올지 먼저 예상하고 실행한다. 이어서 ” 0”, “0”, 매우 긴 숫자를 넣어 문자열 문법과 숫자 범위가 서로 다른 검사임을 확인한다. 허용 각도를 바꿀 때는 정상값과 두 경계 밖 값을 함께 바꿔야 한다. 범위를 넓혀 모든 검사를 통과시키는 것은 버그 수정이 아니다.

ID가 같고 내용이 다른 두 요청을 처리할 정책도 글로 적어 보자. 이 예제는 ID 중복을 검사하지 않는다. 문자열을 유효한 명령으로 구성하는 역할만 맡기 때문이다. 중복과 순서는 통신 계층의 세션·응답 설계에서 추가한다. 작은 함수의 완료 범위를 정확히 이해하는 것이 큰 시스템을 만드는 출발점이다.

실습: 경계를 하나씩 바꾼다

앞의 3.6절 전체 코드를 ~/robot-dev/sample/control.c에 저장한 다음 Ubuntu/WSL Bash에서 PC용 컴파일러로 실행한다.

```
cd ~/robot-dev/sample
mkdir -p /tmp/robot-book-ch03
cc -std=c11 -Wall -Wextra -Werror \
    control.c -o /tmp/robot-book-ch03/control
/tmp/robot-book-ch03/control
```

정상이면 boundary and stop checks passed가 표시된다. 29·30·150·151을 각각 넣었을 때 수락 여부를 실행 전에 적어 보자. STOPPED를 IDLE로 바꾸는 해제 함수를 추가하되, 해제만으로 동작을 시작하지 않게 한다.

컴파일 오류는 처음 보고된 줄부터 찾는다. 경고를 없애기 위한 임의 캐스트로 범위 오류를 숨기지 않는다. assert가 실패하면 입력과 변경 전후 상태를 비교한다. 거부해야 할 입력을 정상 값으로 바꾸어 시험을 통과시키지 않는다.

AI와 함께 문제 해결하기

이 장의 C 예제를 AI와 검토하되, 프로그램의 계약은 독자가 먼저 정한다. “컴파일된다”와 “실패할 때도 약속한 상태를 유지한다”를 서로 다른 질문으로 다루자.

문제 1. 실패 시 상태 보존을 시험하기

입력의 첫 필드를 읽은 직후 출력 구조체를 바꾸는 수정안을 검토해 보자. 올바른 입력과 중간에 실패하는 입력을 직접 고르고 기존 코드와 비교한다.

함께 제공할 자료: ch03_command.c, 자신이 작성한 입력별 예상 상태, 수정안이 있다면 변경 부분.

예시 프롬프트

첨부한 ch03_command.c의 입력 검증 계약을 검토해 줘.
 내가 예상한 반환값과 변경 전후 상태: [예상표]
 실패 시 상태 보존을 깨뜨릴 입력을 내가 찾도록 질문해 줘.
 내 답변 뒤 최소 비교 시험을 제안하고 필요한 변경만 설명해 줘.
 원본 대신 실습용 복사본에서 비교하도록 해 줘.
 실행하지 않았다면 시험 통과라고 쓰지 마.

직접 확인하고 남길 것: 반환값뿐 아니라 실패 전후 구조체도 비교한다. 자신의 예상과 실제 결과가 달랐던 입력을 설명한다.

문제 2. 숫자처럼 보이는 문자열의 경계 찾기

12x, 빈 문자열, 부호만 있는 문자열, 범위를 벗어난 숫자를 후보로 삼아 입력 계약을 정의한다. AI가 만든 시험 목록에 빠진 경계가 있는지 찾아 보자.

함께 제공할 자료: 현재 파싱 코드, 허용 범위, 자신이 분류한 허용·거절 입력 목록.

예시 프롬프트

이 C 코드가 받아들이는 문자열의 규칙을 명확히 하려 한다.
 코드·허용 범위·내 분류: [자료]
 내 분류의 모호함을 먼저 질문해 줘.
 확정한 규칙으로 정상·경계·비정상 시험 입력을 만들어 줘.
 정답표를 먼저 보여 주지 말고 내가 예측한 뒤 코드와 대조해 줘.
 숫자 변환 성공과 문자열 전체 검증을 각각 어디서 확인할지 짚어 줘.

직접 확인하고 남길 것: 각 입력이 어떤 규칙을 시험하는지 적는다. AI가 제안한 입력을 실제 파싱 함수에 넣어 판정을 확인한다.

문제 3. C와 Rust의 데이터 수명과 실행 환경 비교하기

C 함수 안의 지역 배열을 나중에 사용할 때 생기는 수명 문제를 먼저 시간선으로 그려 보자. 그다음 Rust에서는 같은 요구를 어떤 소유권과 빌림 관계로 표현할지 AI와 검토한다. Embassy와 Microduck의 공식 파일도 함께 읽어 언어가 같아도 실행 환경은 달라질 수 있음을 확인한다.

함께 제공할 자료: 짧은 C 코드와 자신의 수명 시간선, Rust 소유권 문서, 본문의 Embassy 예제와 Microduck 빌드 설정 URL. Rust 문법을 모르면 개념 비교부터 진행한다.

예시 프롬프트

C 코드·데이터 수명 시간선·공식 문서: [자료]
 누가 데이터를 만들고 언제까지 사용하는지 먼저 질문해 줘.
 내 답변 뒤 복사·소유권 이전·빌림의 차이를 Rust 관점과 비교해 줘.
 언어가 검사할 수 있는 오류와 관절 범위·정지 정책을 구분해 줘.
 Embassy 예제와 Microduck 설정에서 대상 CPU·OS 근거를 찾아 줘.
 Rust라는 이유만으로 같은 보드에서 실행된다고 결론 내리지 마.
 코드를 제한하면 PC용인지 MCU용인지와 미검증 부분을 표시해 줘.

직접 확인하고 남길 것: 생성·전달·사용·소멸 시점을 표시하고, Rust의 규칙이 어떤 문제를 드러내는 지 자신의 말로 적는다. 두 프로젝트의 대상 CPU·운영체제·드라이버를 출처와 함께 비교한다. 소스 조사와 실제 빌드·보드 실행을 별도 상태로 남긴다.

3장 마무리

실습 완료 체크리스트

- ☐ C와 Rust의 선택 가능성과 이번 프로젝트의 C 사용 이유를 설명했다.
- ☐ MCU Rust 펌웨어와 Microduck의 Linux 기반 제어 소프트웨어를 구분했다.
- ☐ 경고를 켜고 PC 예제를 컴파일했다.
- ☐ 경계·범위 밖·정지 중 요청을 확인했다.
- ☐ 단위와 포인터 수명을 설명할 수 있다.

핵심 정리

C와 Rust 모두 임베디드 개발의 선택지다. 이 프로젝트는 C로 구현하며, Rust는 공식 MCU 예제와 Microduck 같은 제품의 내장 소프트웨어에서도 활용을 확인할 수 있다. 언어가 달라도 제어 코드는 어떤 값을 받아 어떤 상태에서 무엇을 바꾸는지의 약속이다. 다음 장에서는 코드를 보드 실행 파일로 바꾸고 실행 결과를 확인한다.

참고 자료

- **기준 app_state.h**
https://github.com/freshmea/micro_ros_pico_dev/blob/393f96b6e484b5c70245b821ddddd3aca2d7cc1fc/src/app_state.h
- **C11 공개 위원회 초안**
<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>

-
- **Rust 임베디드 공식 안내**
<https://rust-lang.org/what/embedded/>
 - **Embedded Rust Book**
<https://docs.rust-embedded.org/book/intro/index.html>
 - **Embassy RP235x HAL**
<https://docs.embassy.dev/embassy-rp/git/rp235xa/index.html>
 - **Microduck 공식 제품**
<https://pollen-robotics.com/microduck/>
 - **Microduck 공식 소스 — 조사 커밋**
<https://github.com/pollen-robotics/microduck/tree/f0d934e761a0bc96a6a7d1b5bc1260ce5a4120e5>